Deadlock Simulator (C Backend)

Generated by Doxygen 1.14.0

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

DeadlockSet . . . . . ??
Detector . . . . . ??
Engine . . . . . ??
Event . . . . . ??
EventList . . . . . ??
Metrics . . . . . ??
Process . . . . . ??
RequestPolicy . . . . . ??
Resolver . . . . . ??
Resource . . . . . ??
SystemState . . . . . ??

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

src/main.c . . . . . ??
src/core/engine.c . . . . . ??
src/core/engine.h . . . . . ??
src/core/metrics.c . . . . . ??
src/core/metrics.h . . . . . ??
src/core/model.c . . . . . ??
src/core/model.h . . . . . ??
src/core/util.c . . . . . ??
src/core/util.h . . . . . ??
src/core/workload.c . . . . . ??
src/core/workload.h . . . . . ??
src/policies/avoidance_nohold.c . . . . . ??
src/policies/avoidance_nohold.h . . . . . ??
src/policies/avoidance_ordering.c . . . . . ??
src/policies/avoidance_ordering.h . . . . . ??
src/policies/avoidance_preempt.c . . . . . ??
src/policies/avoidance_preempt.h . . . . . ??
src/policies/detector_matrix.c . . . . . ??
src/policies/detector_matrix.h . . . . . ??
src/policies/detector_wfg.c . . . . . ??
src/policies/detector_wfg.h . . . . . ??
src/policies/policy.h . . . . . ??
src/policies/prevention_banker.c . . . . . ??
src/policies/prevention_banker.h . . . . . ??
src/policies/resolver_abort.c . . . . . ??
src/policies/resolver_abort.h . . . . . ??
src/policies/resolver_rollback.c . . . . . ??
src/policies/resolver_rollback.h . . . . . ??

Chapter 3

Class Documentation

3.1 DeadlockSet Struct Reference

```
#include <policy.h>
```

Collaboration diagram for DeadlockSet:

Public Attributes

- int pids [MAX_PROCS]
- int len

3.1.1 Detailed Description

Definition at line 8 of file policy.h.

3.1.2 Member Data Documentation

3.1.2.1 len

```
int DeadlockSet::len
```

Definition at line 10 of file policy.h.

3.1.2.2 pids

```
int DeadlockSet::pids[MAX_PROCS]
```

Definition at line 9 of file policy.h.

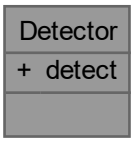
The documentation for this struct was generated from the following file:

- src/policies/policy.h

3.2 Detector Struct Reference

```
#include <policy.h>
```

Collaboration diagram for Detector:



Public Attributes

- int(∗ detect)(const SystemState ∗s, DeadlockSet ∗out_sets, int max_sets)

3.2.1 Detailed Description

Definition at line 13 of file policy.h.

3.2.2 Member Data Documentation

3.2.2.1 detect

```
int(* Detector::detect) (const SystemState *s, DeadlockSet *out_sets, int max_sets)
```

Definition at line 14 of file policy.h.

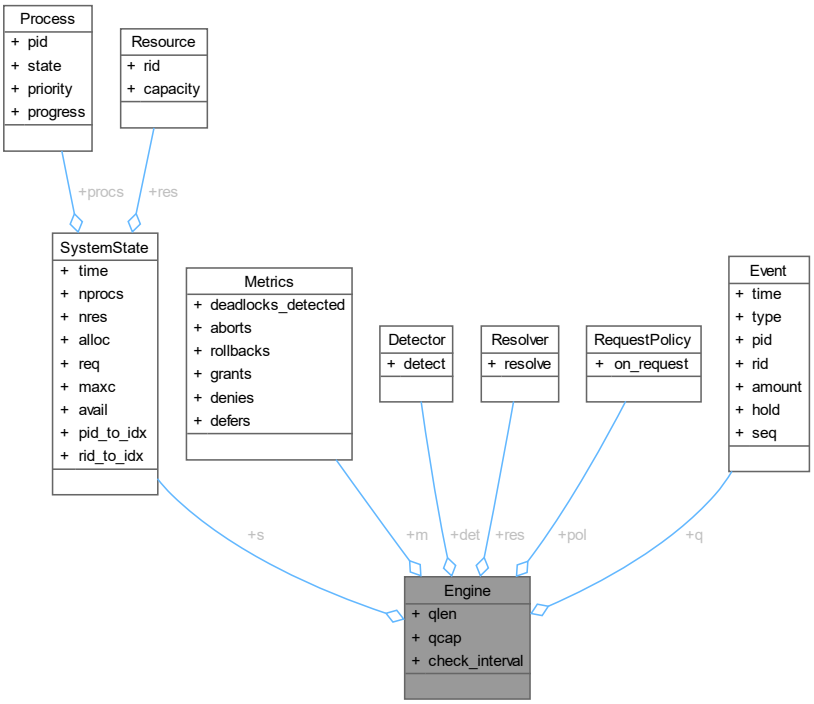
The documentation for this struct was generated from the following file:

- src/policies/policy.h

3.3 Engine Struct Reference

```
#include <engine.h>
```

Collaboration diagram for Engine:



Public Attributes

- SystemState s
- Metrics m
- Detector det
- Resolver res
- RequestPolicy pol
- Event * q
- int qlen
- int qcap
- int check_interval

3.3.1 Detailed Description

Definition at line 7 of file engine.h.

3.3.2 Member Data Documentation

3.3.2.1 check_interval

```
int Engine::check_interval
```

Definition at line 18 of file engine.h.

3.3.2.2 det

```
Detector Engine::det
```

Definition at line 10 of file engine.h.

3.3.2.3 m

```
Metrics Engine::m
```

Definition at line 9 of file engine.h.

3.3.2.4 pol

```
RequestPolicy Engine::pol
```

Definition at line 12 of file engine.h.

3.3.2.5 q

```
Event* Engine::q
```

Definition at line 14 of file engine.h.

3.3.2.6 qcap

```
int Engine::qcap
```

Definition at line 16 of file engine.h.

3.3.2.7 qlen

```
int Engine::qlen
```

Definition at line 15 of file engine.h.

3.3.2.8 res

```
Resolver Engine::res
```

Definition at line 11 of file engine.h.

3.3.2.9 s

```
SystemState Engine::s
```

Definition at line 8 of file engine.h.

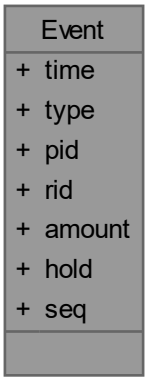
The documentation for this struct was generated from the following file:

- src/core/engine.h

3.4 Event Struct Reference

```
#include <workload.h>
```

Collaboration diagram for Event:



Public Attributes

- int time
- EventType type
- int pid
- int rid
- int amount
- int hold
- int seq

3.4.1 Detailed Description

Definition at line 6 of file workload.h.

3.4.2 Member Data Documentation

3.4.2.1 amount

```
int Event::amount
```

Definition at line 11 of file workload.h.

3.4.2.2 hold

```
int Event::hold
```

Definition at line 12 of file workload.h.

3.4.2.3 pid

```
int Event::pid
```

Definition at line 9 of file workload.h.

3.4.2.4 rid

```
int Event::rid
```

Definition at line 10 of file workload.h.

3.4.2.5 seq

```
int Event::seq
```

Definition at line 13 of file workload.h.

3.4.2.6 time

```
int Event::time
```

Definition at line 7 of file workload.h.

3.4.2.7 type

```
EventType Event::type
```

Definition at line 8 of file workload.h.

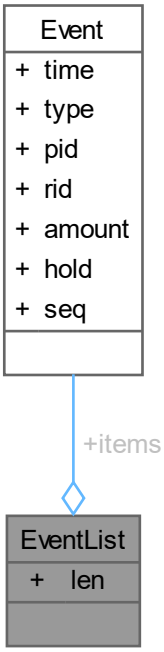
The documentation for this struct was generated from the following file:

- src/core/workload.h

3.5 EventList Struct Reference

```
#include <workload.h>
```

Collaboration diagram for EventList:



Public Attributes

- Event * items
- size_t len

3.5.1 Detailed Description

Definition at line 16 of file workload.h.

3.5.2 Member Data Documentation

3.5.2.1 items

```
Event* EventList::items
```

Definition at line 17 of file workload.h.

3.5.2.2 len

```
size_t EventList::len
```

Definition at line 18 of file workload.h.

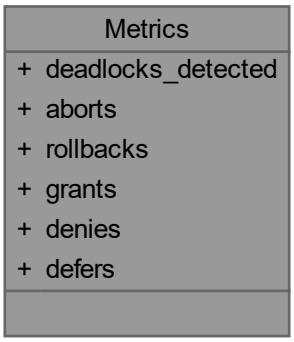
The documentation for this struct was generated from the following file:

- src/core/workload.h

3.6 Metrics Struct Reference

```
#include <metrics.h>
```

Collaboration diagram for Metrics:



Public Attributes

- int deadlocks_detected
- int aborts
- int rollbacks
- int grants
- int denies
- int defers

3.6.1 Detailed Description

Definition at line 4 of file metrics.h.

3.6.2 Member Data Documentation

3.6.2.1 aborts

```
int Metrics::aborts
```

Definition at line 6 of file metrics.h.

3.6.2.2 deadlocks_detected

```
int Metrics::deadlocks_detected
```

Definition at line 5 of file metrics.h.

3.6.2.3 defers

```
int Metrics::defers
```

Definition at line 10 of file metrics.h.

3.6.2.4 denies

```
int Metrics::denies
```

Definition at line 9 of file metrics.h.

3.6.2.5 grants

```
int Metrics::grants
```

Definition at line 8 of file metrics.h.

3.6.2.6 rollbacks

```
int Metrics::rollbacks
```

Definition at line 7 of file metrics.h.

The documentation for this struct was generated from the following file:

- src/core/metrics.h

3.7 Process Struct Reference

```
#include <model.h>
```

Collaboration diagram for Process:

Public Attributes

- int pid
- ProcState state
- int priority
- int progress

3.7.1 Detailed Description

Definition at line 9 of file model.h.

3.7.2 Member Data Documentation

3.7.2.1 pid

```
int Process::pid
```

Definition at line 10 of file model.h.

3.7.2.2 priority

```
int Process::priority
```

Definition at line 12 of file model.h.

3.7.2.3 progress

```
int Process::progress
```

Definition at line 13 of file model.h.

3.7.2.4 state

```
ProcState Process::state
```

Definition at line 11 of file model.h.

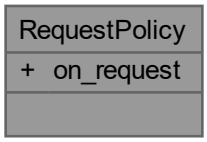
The documentation for this struct was generated from the following file:

- src/core/model.h

3.8 RequestPolicy Struct Reference

```
#include <policy.h>
```

Collaboration diagram for RequestPolicy:



Public Attributes

- Decision(∗ on_request)(const Event ∗e, SystemState ∗s, Metrics ∗m)

3.8.1 Detailed Description

Definition at line 21 of file policy.h.

3.8.2 Member Data Documentation

3.8.2.1 on_request

```
Decision(* RequestPolicy::on_request) (const Event *e, SystemState *s, Metrics *m)
```

Definition at line 22 of file policy.h.

The documentation for this struct was generated from the following file:

- src/policies/policy.h

3.9 Resolver Struct Reference

```
#include <policy.h>
```

Collaboration diagram for Resolver:

Public Attributes

- void(∗ resolve)(const DeadlockSet ∗d, SystemState ∗s, int now, Metrics ∗m)

3.9.1 Detailed Description

Definition at line 17 of file policy.h.

3.9.2 Member Data Documentation

3.9.2.1 resolve

```
void(* Resolver::resolve) (const DeadlockSet *d, SystemState *s, int now, Metrics *m)
```

Definition at line 18 of file policy.h.

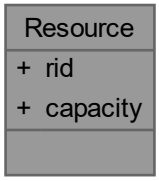
The documentation for this struct was generated from the following file:

- src/policies/policy.h

3.10 Resource Struct Reference

```
#include <model.h>
```

Collaboration diagram for Resource:



Public Attributes

- int rid
- int capacity

3.10.1 Detailed Description

Definition at line 16 of file model.h.

3.10.2 Member Data Documentation

3.10.2.1 capacity

```
int Resource::capacity
```

Definition at line 18 of file model.h.

3.10.2.2 rid

```
int Resource::rid
```

Definition at line 17 of file model.h.

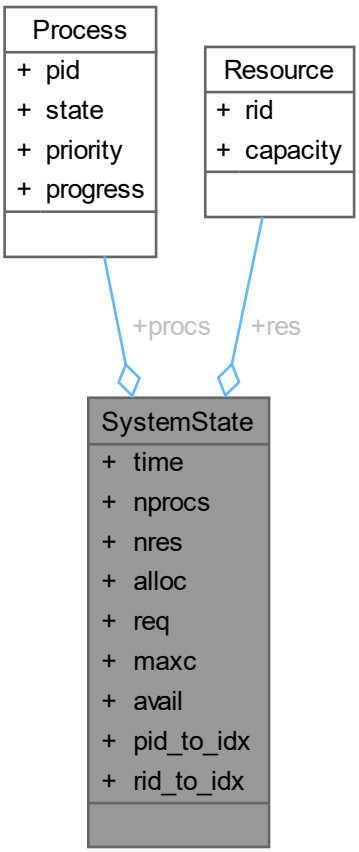
The documentation for this struct was generated from the following file:

- src/core/model.h

3.11 SystemState Struct Reference

```
#include <model.h>
```

Collaboration diagram for SystemState:



Public Attributes

- int time
- int nprocs
- int nres
- Process procs [MAX_PROCS]
- Resource res [MAX_RES]
- int alloc [MAX_PROCS][MAX_RES]
- int req [MAX_PROCS][MAX_RES]
- int maxc [MAX_PROCS][MAX_RES]
- int avail [MAX_RES]
- int pid_to_idx [MAX_PROCS]
- int rid_to_idx [MAX_RES]

3.11.1 Detailed Description

Definition at line 21 of file model.h.

3.11.2 Member Data Documentation

3.11.2.1 alloc

```
int SystemState::alloc[MAX_PROCS][MAX_RES]
```

Definition at line 29 of file model.h.

3.11.2.2 avail

```
int SystemState::avail[MAX_RES]
```

Definition at line 32 of file model.h.

3.11.2.3 maxc

```
int SystemState::maxc[MAX_PROCS][MAX_RES]
```

Definition at line 31 of file model.h.

3.11.2.4 nprocs

```
int SystemState::nprocs
```

Definition at line 23 of file model.h.

3.11.2.5 nres

```
int SystemState::nres
```

Definition at line 24 of file model.h.

3.11.2.6 pid_to_idx

```
int SystemState::pid_to_idx[MAX_PROCS]
```

Definition at line 34 of file model.h.

3.11.2.7 procs

```
Process SystemState::procs[MAX_PROCS]
```

Definition at line 26 of file model.h.

3.11.2.8 req

```
int SystemState::req[MAX_PROCS][MAX_RES]
```

Definition at line 30 of file model.h.

3.11.2.9 res

```
Resource SystemState::res[MAX_RES]
```

Definition at line 27 of file model.h.

3.11.2.10 rid_to_idx

```
int SystemState::rid_to_idx[MAX_RES]
```

Definition at line 35 of file model.h.

3.11.2.11 time

```
int SystemState::time
```

Definition at line 22 of file model.h.

The documentation for this struct was generated from the following file:

- src/core/model.h

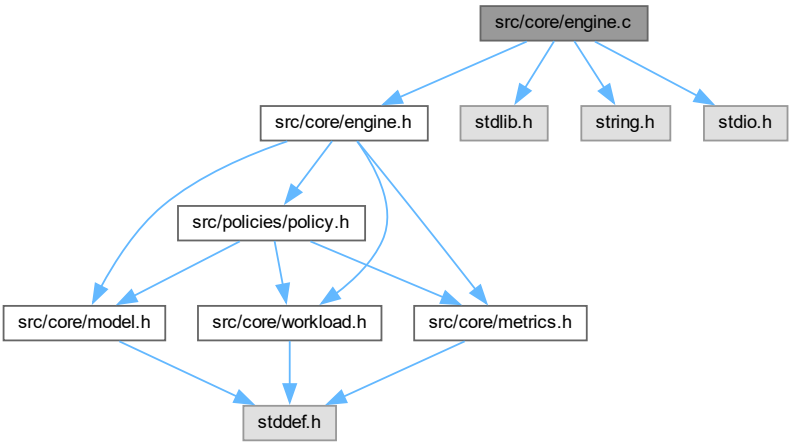
Chapter 4

File Documentation

4.1 src/core/engine.c File Reference

```
#include "src/core/engine.h"
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
```

Include dependency graph for engine.c:



Functions

- static int cmp_ev (const Event ∗a, const Event ∗b)
- void engine_init (Engine ∗e)
- void engine_set_detector (Engine ∗e, Detector d)
- void engine_set_resolver (Engine ∗e, Resolver r)
- void engine_set_policy (Engine ∗e, RequestPolicy p)
- void engine_add_event (Engine ∗e, Event ev)
- void engine_load_events (Engine ∗e, const EventList ∗list)

- static void sort_queue (Engine ∗e)
- static void do_deadlock_check (Engine ∗e, int now)
- static void handle_arrive (Engine ∗e, const Event ∗ev)
- static void handle_finish (Engine ∗e, const Event ∗ev)
- static void handle_rel (Engine ∗e, const Event ∗ev)
- static void handle_req (Engine ∗e, const Event ∗ev)
- static void try_unblock (Engine ∗e)
- void engine_run (Engine ∗e, int until_time)

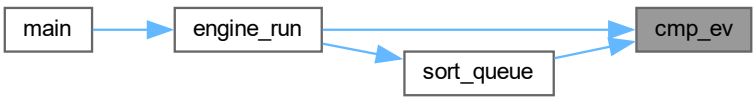
4.1.1 Function Documentation

4.1.1.1 cmp_ev()

```
int cmp_ev (
            const Event * a,
            const Event * b)  [static]
```

Definition at line 6 of file engine.c.

Here is the caller graph for this function:

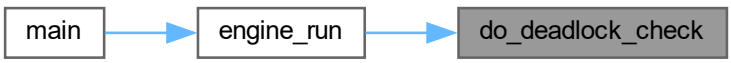


4.1.1.2 do_deadlock_check()

```
void do_deadlock_check (
            Engine * e,
            int now)  [static]
```

Definition at line 49 of file engine.c.

Here is the caller graph for this function:

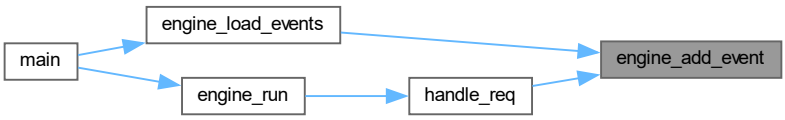


4.1.1.3 engine_add_event()

```
void engine_add_event (
            Engine * e,
            Event ev)
```

Definition at line 25 of file engine.c.

Here is the caller graph for this function:

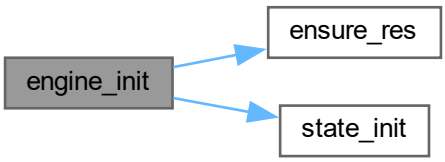


4.1.1.4 engine_init()

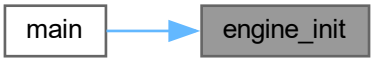
```
void engine_init (
            Engine * e)
```

Definition at line 11 of file engine.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.1.5 engine_load_events()

```
void engine_load_events (
            Engine * e,
            const EventList * list)
```

Definition at line 33 of file engine.c.

Here is the call graph for this function:

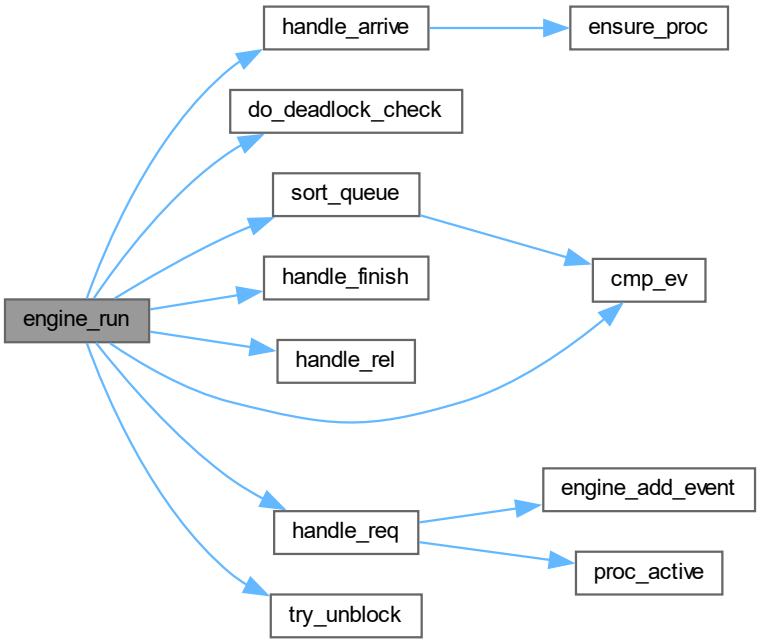
Here is the caller graph for this function:

4.1.1.6 engine_run()

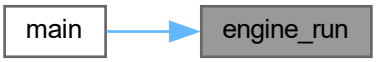
```
void engine_run (
            Engine * e,
            int until_time)
```

Definition at line 149 of file engine.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.1.7 engine_set_detector()

```
void engine_set_detector (
            Engine * e,
            Detector d)
```

Definition at line 21 of file engine.c.

Here is the caller graph for this function:

4.1.1.8 engine_set_policy()

```
void engine_set_policy (
            Engine * e,
            RequestPolicy p)
```

Definition at line 23 of file engine.c.

Here is the caller graph for this function:

4.1.1.9 engine_set_resolver()

```
void engine_set_resolver (
            Engine * e,
            Resolver r)
```

Definition at line 22 of file engine.c.

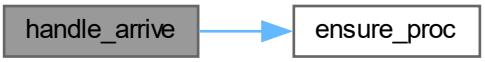
Here is the caller graph for this function:

4.1.1.10 handle_arrive()

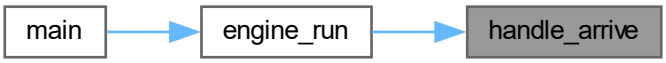
```
void handle_arrive (
            Engine * e,
            const Event * ev)  [static]
```

Definition at line 57 of file engine.c.

Here is the call graph for this function:



Here is the caller graph for this function:

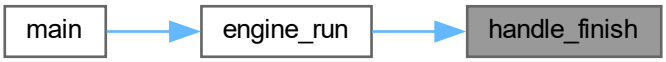


4.1.1.11 handle_finish()

```
void handle_finish (
            Engine * e,
            const Event * ev)  [static]
```

Definition at line 65 of file engine.c.

Here is the caller graph for this function:

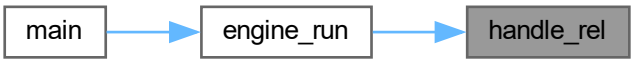


4.1.1.12 handle_rel()

```
void handle_rel (
            Engine * e,
            const Event * ev)  [static]
```

Definition at line 79 of file engine.c.

Here is the caller graph for this function:

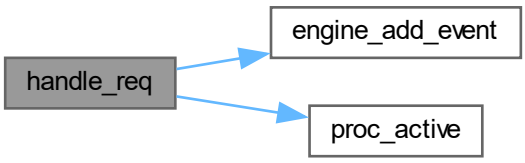


4.1.1.13 handle_req()

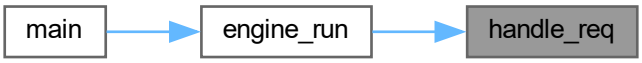
```
void handle_req (
            Engine * e,
            const Event * ev)  [static]
```

Definition at line 90 of file engine.c.

Here is the call graph for this function:



Here is the caller graph for this function:



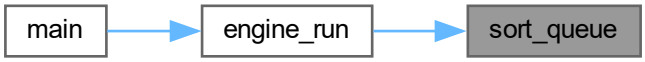
4.1.1.14 sort_queue()

```
void sort_queue (
            Engine * e)  [static]
```

Definition at line 37 of file engine.c.

Here is the call graph for this function:

Here is the caller graph for this function:

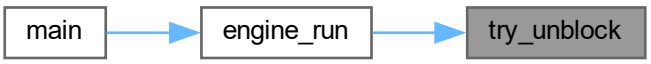


4.1.1.15 try_unblock()

```
void try_unblock (
            Engine * e)  [static]
```

Definition at line 128 of file engine.c.

Here is the caller graph for this function:



4.2 engine.c

Go to the documentation of this file.

```c
#include "src/core/engine.h"
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

static int cmp_ev(const Event* a, const Event* b) {
  if (a->time != b->time) return (a->time < b->time) ? -1 : 1;
  return (a->seq < b->seq) ? -1 : (a->seq > b->seq);
}

void engine_init(Engine* e) {
  memset(e, 0, sizeof(*e));
  state_init(&e->s);
  e->qcap = 256;
  e->q = (Event*)malloc(sizeof(Event)*e->qcap);
  e->check_interval = 1;
  ensure_res(&e->s, 0, 1);
  ensure_res(&e->s, 1, 1);
}

void engine_set_detector(Engine* e, Detector d) { e->det = d; }
void engine_set_resolver(Engine* e, Resolver r) { e->res = r; }
void engine_set_policy(Engine* e, RequestPolicy p) { e->pol = p; }

void engine_add_event(Engine* e, Event ev) {
  if (e->qlen == e->qcap) {
    e->qcap *= 2;
    e->q = (Event*)realloc(e->q, sizeof(Event)*e->qcap);
  }
  e->q[e->qlen++] = ev;
}

void engine_load_events(Engine* e, const EventList* list) {
  for (size_t i=0;i<list->len;i++) engine_add_event(e, list->items[i]);
}

static void sort_queue(Engine* e) {
  for (int i=0;i<e->qlen;i++) {
    for (int j=i+1;j<e->qlen;j++) {
      if (cmp_ev(&e->q[j], &e->q[i]) < 0) {
        Event tmp = e->q[i];
        e->q[i] = e->q[j];
        e->q[j] = tmp;
      }
    }
  }
}

static void do_deadlock_check(Engine* e, int now) {
  DeadlockSet sets[8];
  int n = 0;
  if (e->det.detect) n = e->det.detect(&e->s, sets, 8);
  if (n > 0) e->m.deadlocks_detected += n;
  for (int i=0;i<n;i++) if (e->res.resolve) e->res.resolve(&sets[i], &e->s, now, &e->m);
}

static void handle_arrive(Engine* e, const Event* ev) {
  int pi = ensure_proc(&e->s, ev->pid);
  e->s.procs[pi].state = PS_READY;
  for (int r=0;r<e->s.nres;r++) {
    if (e->s.maxc[pi][r] == 0) e->s.maxc[pi][r] = e->s.res[r].capacity;
  }
}

static void handle_finish(Engine* e, const Event* ev) {
  for (int i=0;i<e->s.nprocs;i++) {
    if (e->s.procs[i].pid == ev->pid) {
      e->s.procs[i].state = PS_FINISHED;
      for (int r=0;r<e->s.nres;r++) {
        e->s.avail[r] += e->s.alloc[i][r];
        e->s.alloc[i][r] = 0;
        e->s.req[i][r] = 0;
      }
      break;
    }
  }
}

static void handle_rel(Engine* e, const Event* ev) {
  int pi=-1, ri=-1;
  for (int i=0;i<e->s.nprocs;i++) if (e->s.procs[i].pid == ev->pid) { pi=i; break; }
  for (int r=0;r<e->s.nres;r++) if (e->s.res[r].rid == ev->rid) { ri=r; break; }
```

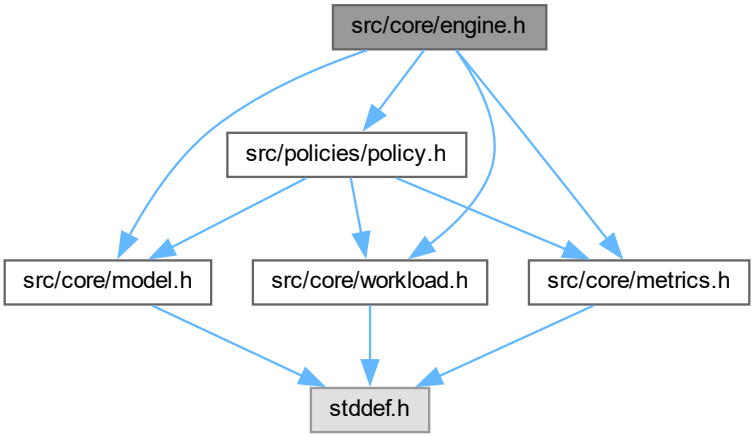
```
  if (pi<0 || ri<0) return;
  int k = ev->amount;
  if (k > e->s.alloc[pi][ri]) k = e->s.alloc[pi][ri];
  e->s.alloc[pi][ri] -= k;
  e->s.avail[ri] += k;
}

static void handle_req(Engine* e, const Event* ev) {
  int pi=-1, ri=-1;
  for (int i=0;i<e->s.nprocs;i++) if (e->s.procs[i].pid == ev->pid) { pi=i; break; }
  for (int r=0;r<e->s.nres;r++) if (e->s.res[r].rid == ev->rid) { ri=r; break; }
  if (pi<0 || ri<0) return;
  if (!proc_active(e->s.procs[pi].state)) return;

  Decision d = DEC_DEFER;
  if (e->pol.on_request) d = e->pol.on_request(ev, &e->s, &e->m);
  else {
    if (ev->amount <= e->s.avail[ri]) {
      e->s.avail[ri] -= ev->amount;
      e->s.alloc[pi][ri] += ev->amount;
      e->m.grants++;
      d = DEC_GRANT;
    } else {
      e->m.defers++;
      d = DEC_DEFER;
    }
  }

  if (d == DEC_DEFER) {
    e->s.req[pi][ri] += ev->amount;
    e->s.procs[pi].state = PS_BLOCKED;
  } else if (d == DEC_DENY) {
    e->s.procs[pi].state = PS_BLOCKED;
  } else {
    e->s.procs[pi].state = PS_READY;
    if (ev->hold > 0) {
      Event rel = *ev;
      rel.type = EV_REL;
      rel.time = ev->time + ev->hold;
      rel.hold = 0;
      engine_add_event(e, rel);
    }
  }
}

static void try_unblock(Engine* e) {
  for (int pi=0; pi<e->s.nprocs; pi++) {
    if (e->s.procs[pi].state != PS_BLOCKED) continue;
    int unblocked = 1;
    for (int ri=0; ri<e->s.nres; ri++) {
      int need = e->s.req[pi][ri];
      if (need <= 0) continue;
      if (need > e->s.avail[ri]) { unblocked = 0; break; }
    }
    if (!unblocked) continue;
    for (int ri=0; ri<e->s.nres; ri++) {
      int need = e->s.req[pi][ri];
      if (need <= 0) continue;
      e->s.avail[ri] -= need;
      e->s.alloc[pi][ri] += need;
      e->s.req[pi][ri] = 0;
    }
    e->s.procs[pi].state = PS_READY;
  }
}

void engine_run(Engine* e, int until_time) {
  sort_queue(e);

  int idx = 0;
  while (idx < e->qlen) {
    Event ev = e->q[idx++];
    if (ev.time > until_time) break;
    e->s.time = ev.time;

    if (ev.type == EV_ARRIVE) handle_arrive(e, &ev);
    else if (ev.type == EV_REQ) handle_req(e, &ev);
    else if (ev.type == EV_REL) handle_rel(e, &ev);
    else if (ev.type == EV_FINISH) handle_finish(e, &ev);
    else if (ev.type == EV_CHECK) do_deadlock_check(e, ev.time);

    if (e->check_interval > 0) do_deadlock_check(e, ev.time);
    try_unblock(e);

    if (idx < e->qlen) {
      for (int i=idx;i<e->qlen;i++) {
        for (int j=i+1;j<e->qlen;j++) {
```

```
            if (cmp_ev(&e->q[j], &e->q[i]) < 0) {
              Event tmp = e->q[i]; e->q[i] = e->q[j]; e->q[j] = tmp;
            }
          }
        }
      }
    }
}
```

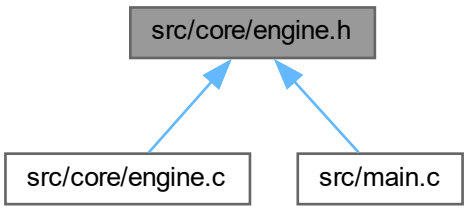
4.3 src/core/engine.h File Reference

```
#include "src/core/model.h"
#include "src/core/workload.h"
#include "src/core/metrics.h"
#include "src/policies/policy.h"
```

Include dependency graph for engine.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct Engine

Functions

- void engine_init (Engine ∗e)
- void engine_set_detector (Engine ∗e, Detector d)
- void engine_set_resolver (Engine ∗e, Resolver r)
- void engine_set_policy (Engine ∗e, RequestPolicy p)
- void engine_add_event (Engine ∗e, Event ev)
- void engine_load_events (Engine ∗e, const EventList ∗list)
- void engine_run (Engine ∗e, int until_time)

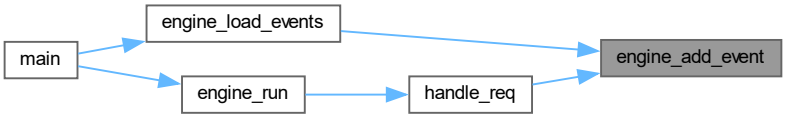
4.3.1 Function Documentation

4.3.1.1 engine_add_event()

```
void engine_add_event (
            Engine * e,
            Event ev)
```

Definition at line 25 of file engine.c.

Here is the caller graph for this function:

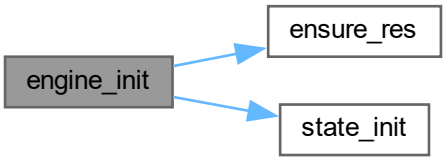


4.3.1.2 engine_init()

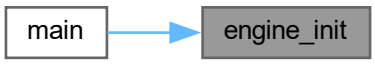
```
void engine_init (
            Engine * e)
```

Definition at line 11 of file engine.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.1.3 engine_load_events()

```
void engine_load_events (
            Engine * e,
            const EventList * list)
```

Definition at line 33 of file engine.c.

Here is the call graph for this function:

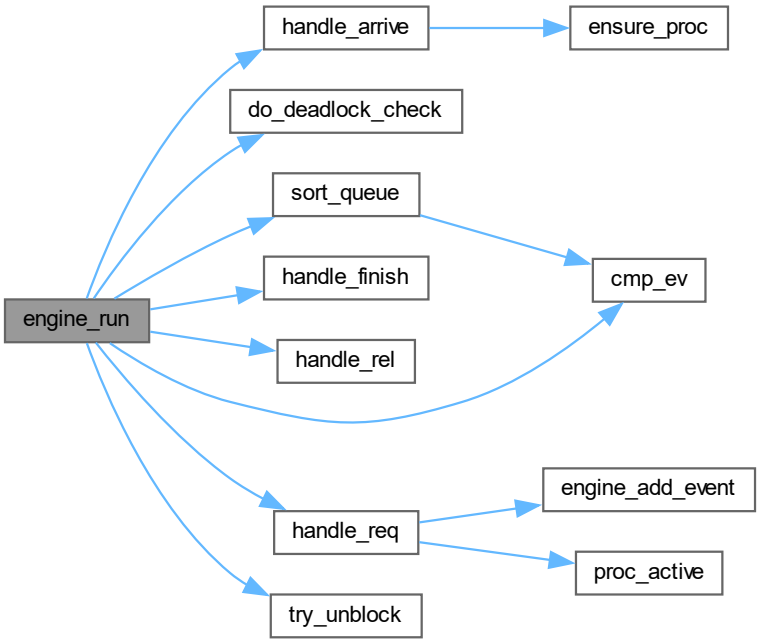
Here is the caller graph for this function:

4.3.1.4 engine_run()

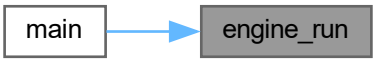
```
void engine_run (
            Engine * e,
            int until_time)
```

Definition at line 149 of file engine.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.1.5 engine_set_detector()

```
void engine_set_detector (
            Engine * e,
            Detector d)
```

Definition at line 21 of file engine.c.

Here is the caller graph for this function:

4.3.1.6 engine_set_policy()

```
void engine_set_policy (
            Engine * e,
            RequestPolicy p)
```

Definition at line 23 of file engine.c.

Here is the caller graph for this function:

4.3.1.7 engine_set_resolver()

```
void engine_set_resolver (
            Engine * e,
            Resolver r)
```

Definition at line 22 of file engine.c.

Here is the caller graph for this function:

4.4 engine.h

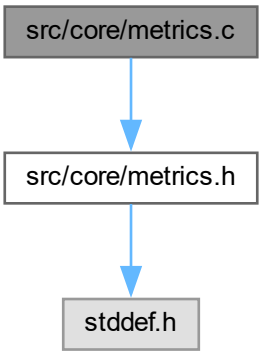
Go to the documentation of this file.

```
#pragma once
#include "src/core/model.h"
#include "src/core/workload.h"
#include "src/core/metrics.h"
#include "src/policies/policy.h"

typedef struct {
  SystemState s;
  Metrics m;
  Detector det;
  Resolver res;
  RequestPolicy pol;

  Event* q;
  int qlen;
  int qcap;

  int check_interval;
} Engine;

void engine_init(Engine* e);
void engine_set_detector(Engine* e, Detector d);
void engine_set_resolver(Engine* e, Resolver r);
void engine_set_policy(Engine* e, RequestPolicy p);

void engine_add_event(Engine* e, Event ev);
void engine_load_events(Engine* e, const EventList* list);

void engine_run(Engine* e, int until_time);
```

4.5 src/core/metrics.c File Reference

`#include "src/core/metrics.h"`

Include dependency graph for metrics.c:



4.6 metrics.c

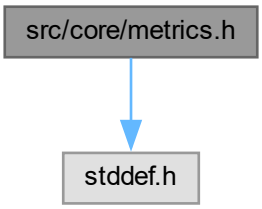
Go to the documentation of this file.

```
#include "src/core/metrics.h"
```

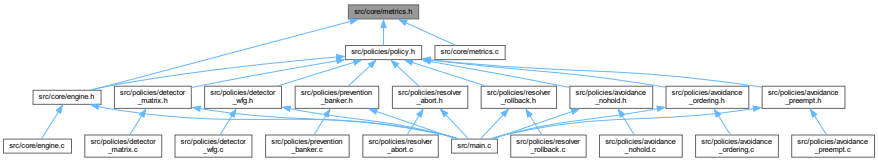
4.7 src/core/metrics.h File Reference

`#include <stddef.h>`

Include dependency graph for metrics.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct Metrics

4.8 metrics.h

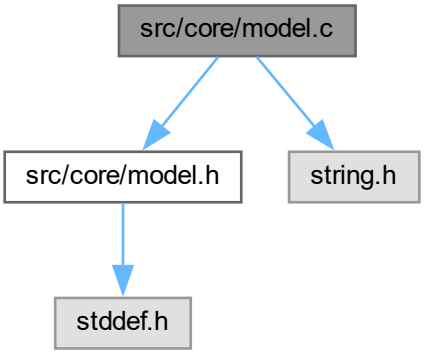
Go to the documentation of this file.

```
#pragma once
#include <stddef.h>

typedef struct {
  int deadlocks_detected;
  int aborts;
  int rollbacks;
  int grants;
  int denies;
  int defers;
} Metrics;
```

4.9 src/core/model.c File Reference

```
#include "src/core/model.h"
#include <string.h>
```

Include dependency graph for model.c:



Functions

- void state_init (SystemState *s)
- int proc_active (ProcState st)
- int ensure_proc (SystemState *s, int pid)
- int ensure_res (SystemState *s, int rid, int capacity)

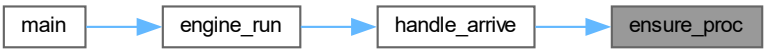
4.9.1 Function Documentation

4.9.1.1 ensure_proc()

```
int ensure_proc (
            SystemState * s,
            int pid)
```

Definition at line 14 of file model.c.

Here is the caller graph for this function:

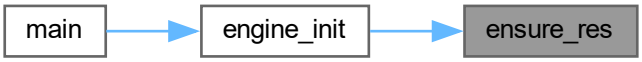


4.9.1.2 ensure_res()

```
int ensure_res (
            SystemState * s,
            int rid,
            int capacity)
```

Definition at line 24 of file model.c.

Here is the caller graph for this function:

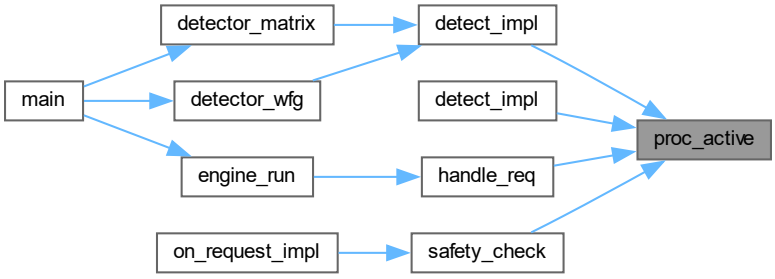


4.9.1.3 proc_active()

```
int proc_active (
            ProcState st)
```

Definition at line 10 of file model.c.

Here is the caller graph for this function:

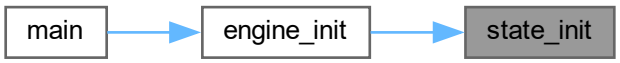


4.9.1.4 state_init()

```
void state_init (
            SystemState * s)
```

Definition at line 4 of file model.c.

Here is the caller graph for this function:



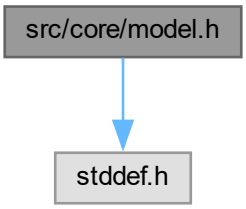
4.10 model.c

Go to the documentation of this file.

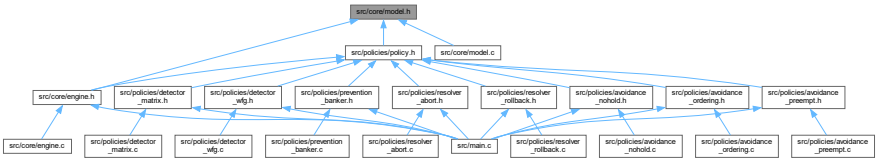
```
00001 #include "src/core/model.h"
00002 #include <string.h>
00003
00004 void state_init(SystemState* s) {
00005   memset(s, 0, sizeof(*s));
00006   for (int i=0;i<MAX_PROCS;i++) s->pid_to_idx[i] = -1;
00007   for (int i=0;i<MAX_RES;i++) s->rid_to_idx[i] = -1;
00008 }
00009
00010 int proc_active(ProcState st) {
00011   return st != PS_FINISHED && st != PS_ABORTED;
00012 }
00013
00014 int ensure_proc(SystemState* s, int pid) {
00015   for (int i=0;i<s->nprocs;i++) if (s->procs[i].pid == pid) return i;
00016   int idx = s->nprocs++;
00017   s->procs[idx].pid = pid;
00018   s->procs[idx].state = PS_NEW;
00019   s->procs[idx].priority = 0;
00020   s->procs[idx].progress = 0;
00021   return idx;
00022 }
00023
00024 int ensure_res(SystemState* s, int rid, int capacity) {
00025   for (int i=0;i<s->nres;i++) if (s->res[i].rid == rid) return i;
00026   int idx = s->nres++;
00027   s->res[idx].rid = rid;
00028   s->res[idx].capacity = capacity;
00029   s->avail[idx] = capacity;
00030   return idx;
00031 }
```

4.11 src/core/model.h File Reference

`#include <stddef.h>`
Include dependency graph for model.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct Process
- struct Resource
- struct SystemState

Macros

- #define MAX_PROCS 128
- #define MAX_RES 64

Enumerations

- enum ProcState {
  PS_NEW , PS_READY , PS_RUNNING , PS_BLOCKED ,
  PS_FINISHED , PS_ABORTED , PS_ROLLED_BACK }

Functions

- void state_init (SystemState ∗s)
- int ensure_proc (SystemState ∗s, int pid)
- int ensure_res (SystemState ∗s, int rid, int capacity)
- int proc_active (ProcState st)

4.11.1 Macro Definition Documentation

4.11.1.1 MAX_PROCS

```
#define MAX_PROCS 128
```

Definition at line 4 of file model.h.

4.11.1.2 MAX_RES

```
#define MAX_RES 64
```

Definition at line 5 of file model.h.

4.11.2 Enumeration Type Documentation

4.11.2.1 ProcState

```
enum ProcState
```

Enumerator

	PS_NEW
	PS_READY
	PS_RUNNING
	PS_BLOCKED
	PS_FINISHED
	PS_ABORTED
	PS_ROLLED_BACK

Definition at line 7 of file model.h.

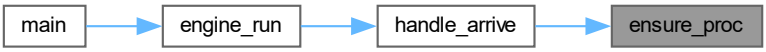
4.11.3 Function Documentation

4.11.3.1 ensure_proc()

```
int ensure_proc (
            SystemState * s,
            int pid)
```

Definition at line 14 of file model.c.

Here is the caller graph for this function:

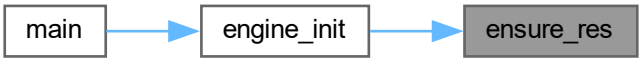


4.11.3.2 ensure_res()

```
int ensure_res (
            SystemState * s,
            int rid,
            int capacity)
```

Definition at line 24 of file model.c.

Here is the caller graph for this function:

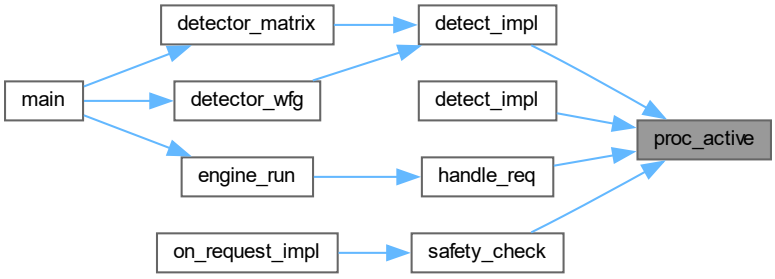


4.11.3.3 proc_active()

```
int proc_active (
            ProcState st)
```

Definition at line 10 of file model.c.

Here is the caller graph for this function:

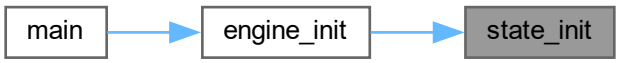


4.11.3.4 state_init()

```
void state_init (
            SystemState * s)
```

Definition at line 4 of file model.c.

Here is the caller graph for this function:



4.12 model.h

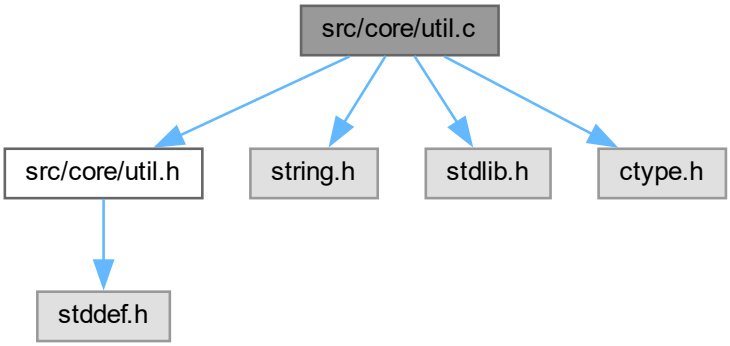
Go to the documentation of this file.

```
00001 #pragma once
00002 #include <stddef.h>
00003
00004 #define MAX_PROCS 128
00005 #define MAX_RES 64
00006
00007 typedef enum { PS_NEW, PS_READY, PS_RUNNING, PS_BLOCKED, PS_FINISHED, PS_ABORTED, PS_ROLLED_BACK }
      ProcState;
00008
00009 typedef struct {
00010   int pid;
00011   ProcState state;
00012   int priority;
00013   int progress;
00014 } Process;
00015
00016 typedef struct {
00017   int rid;
00018   int capacity;
00019 } Resource;
00020
00021 typedef struct {
00022   int time;
00023   int nprocs;
00024   int nres;
00025
00026   Process procs[MAX_PROCS];
00027   Resource res[MAX_RES];
00028
00029   int alloc[MAX_PROCS][MAX_RES];
00030   int req[MAX_PROCS][MAX_RES];
00031   int maxc[MAX_PROCS][MAX_RES];
00032   int avail[MAX_RES];
00033
00034   int pid_to_idx[MAX_PROCS];
00035   int rid_to_idx[MAX_RES];
00036 } SystemState;
00037
00038 void state_init(SystemState* s);
00039 int ensure_proc(SystemState* s, int pid);
00040 int ensure_res(SystemState* s, int rid, int capacity);
00041 int proc_active(ProcState st);
```

4.13 src/core/util.c File Reference

```
#include "src/core/util.h"
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
```

Include dependency graph for util.c:



Functions

- int str_eq (const char ∗a, const char ∗b)
- int to_int (const char ∗s)
- static char ∗ trim (char ∗s)
- size_t split_csv (char ∗line, char ∗∗out, size_t max_out)

4.13.1 Function Documentation

4.13.1.1 split_csv()

```
size_t split_csv (
            char * line,
            char ** out,
            size_t max_out)
```

Definition at line 18 of file util.c.

Here is the call graph for this function:

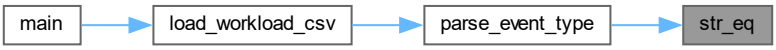
Here is the caller graph for this function:

4.13.1.2 str_eq()

```
int str_eq (
            const char * a,
            const char * b)
```

Definition at line 6 of file util.c.

Here is the caller graph for this function:

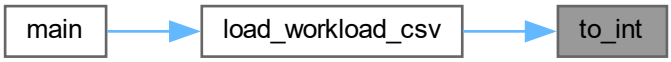


4.13.1.3 to_int()

```
int to_int (
            const char * s)
```

Definition at line 8 of file util.c.

Here is the caller graph for this function:



4.13.1.4 trim()

```
char * trim (
            char * s)  [static]
```

Definition at line 10 of file util.c.

Here is the caller graph for this function:

4.14 util.c

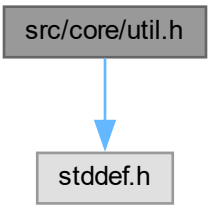
Go to the documentation of this file.

```
#include "src/core/util.h"
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

int str_eq(const char* a, const char* b) { return strcmp(a,b)==0; }

int to_int(const char* s) { return (int)strtol(s, NULL, 10); }

static char* trim(char* s) {
  while (*s && isspace((unsigned char)*s)) s++;
  char* e = s + strlen(s);
  while (e > s && isspace((unsigned char)e[-1])) e--;
  *e = 0;
  return s;
}

size_t split_csv(char* line, char** out, size_t max_out) {
  size_t n = 0;
  char* p = line;
  while (*p && n < max_out) {
    char* start = p;
    while (*p && *p != ',') p++;
    if (*p == ',') { *p = 0; p++; }
    out[n++] = trim(start);
  }
  return n;
}
```

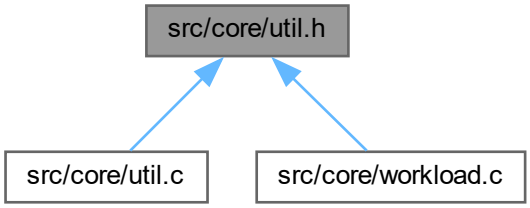
4.15 src/core/util.h File Reference

```
#include <stddef.h>
```

Include dependency graph for util.h:



This graph shows which files directly or indirectly include this file:



Functions

- int str_eq (const char ∗a, const char ∗b)
- int to_int (const char ∗s)
- size_t split_csv (char ∗line, char ∗∗out, size_t max_out)

4.15.1 Function Documentation

4.15.1.1 split_csv()

```
size_t split_csv (
            char * line,
            char ** out,
            size_t max_out)
```

Definition at line 18 of file util.c.

Here is the call graph for this function:

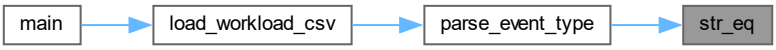
Here is the caller graph for this function:

4.15.1.2 str_eq()

```
int str_eq (
            const char * a,
            const char * b)
```

Definition at line 6 of file util.c.

Here is the caller graph for this function:

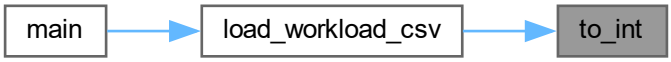


4.15.1.3 to_int()

```
int to_int (
            const char * s)
```

Definition at line 8 of file util.c.

Here is the caller graph for this function:



4.16 util.h

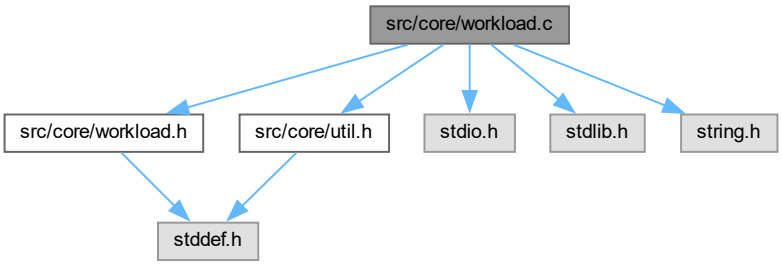
Go to the documentation of this file.

```
00001 #pragma once
00002 #include <stddef.h>
00003
00004 int str_eq(const char* a, const char* b);
00005 int to_int(const char* s);
00006 size_t split_csv(char* line, char** out, size_t max_out);
```

4.17 src/core/workload.c File Reference

```
#include "src/core/workload.h"
#include "src/core/util.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Include dependency graph for workload.c:



Functions

- const char ∗ event_type_name (EventType t)
- int parse_event_type (const char ∗s, EventType ∗out)
- EventList load_workload_csv (const char ∗path)

4.17.1 Function Documentation

4.17.1.1 event_type_name()

```
const char * event_type_name (
            EventType t)
```

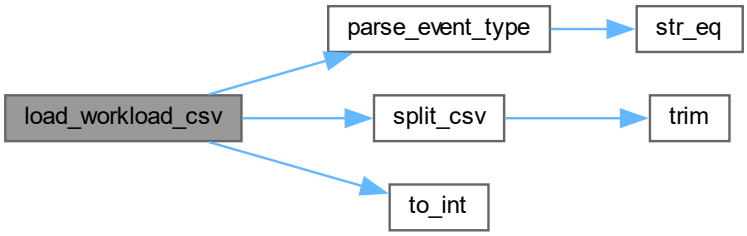
Definition at line 7 of file workload.c.

4.17.1.2 load_workload_csv()

```
EventList load_workload_csv (
            const char * path)
```

Definition at line 27 of file workload.c.

Here is the call graph for this function:



Here is the caller graph for this function:

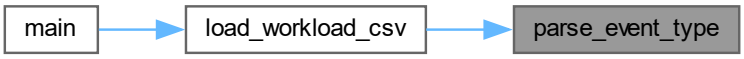
4.17.1.3 parse_event_type()

```
int parse_event_type (
            const char * s,
            EventType * out)
```

Definition at line 18 of file workload.c.

Here is the call graph for this function:

Here is the caller graph for this function:



4.18 workload.c

Go to the documentation of this file.

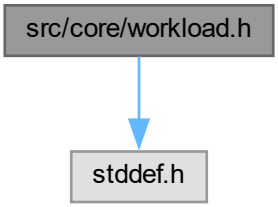
```
00001 #include "src/core/workload.h"
00002 #include "src/core/util.h"
00003 #include <stdio.h>
00004 #include <stdlib.h>
00005 #include <string.h>
00006
00007 const char* event_type_name(EventType t) {
00008   switch (t) {
00009     case EV_ARRIVE: return "ARRIVE";
00010     case EV_REQ: return "REQ";
00011     case EV_REL: return "REL";
00012     case EV_FINISH: return "FINISH";
00013     case EV_CHECK: return "CHECK";
00014   }
00015   return "?";
00016 }
00017
00018 int parse_event_type(const char* s, EventType* out) {
00019   if (str_eq(s,"ARRIVE")) { *out = EV_ARRIVE; return 1; }
00020   if (str_eq(s,"REQ")) { *out = EV_REQ; return 1; }
00021   if (str_eq(s,"REL")) { *out = EV_REL; return 1; }
00022   if (str_eq(s,"FINISH")) { *out = EV_FINISH; return 1; }
00023   if (str_eq(s,"CHECK")) { *out = EV_CHECK; return 1; }
00024   return 0;
00025 }
00026
00027 EventList load_workload_csv(const char* path) {
00028   FILE* f = fopen(path, "r");
00029   if (!f) { fprintf(stderr, "cannot open workload %s\n", path); exit(1); }
```

```

  size_t cap = 256;
  Event* arr = (Event*)malloc(sizeof(Event) * cap);
  size_t len = 0;

  char line[512];
  int seq = 0;
  while (fgets(line, sizeof(line), f)) {
    if (line[0] == '#' || line[0] == '\n' || line[0] == '\r') continue;
    line[strcspn(line, "\r\n")] = 0;

    char* cols[8];
    size_t n = split_csv(line, cols, 8);
    if (n < 6) { fprintf(stderr, "bad line\n"); exit(1); }

    EventType t;
    if (!parse_event_type(cols[1], &t)) { fprintf(stderr, "bad event type\n"); exit(1); }

    if (len == cap) { cap *= 2; arr = (Event*)realloc(arr, sizeof(Event)*cap); }

    arr[len].time = to_int(cols[0]);
    arr[len].type = t;
    arr[len].pid = to_int(cols[2]);
    arr[len].rid = to_int(cols[3]);
    arr[len].amount = to_int(cols[4]);
    arr[len].hold = to_int(cols[5]);
    arr[len].seq = seq++;
    len++;
  }

  fclose(f);
  EventList out = { arr, len };
  return out;
}
```

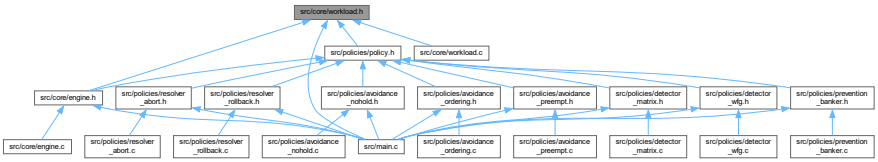
4.19 src/core/workload.h File Reference

`#include <stddef.h>`

Include dependency graph for workload.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct Event
- struct EventList

Enumerations

- enum EventType {
  EV_ARRIVE , EV_REQ , EV_REL , EV_FINISH ,
  EV_CHECK }

Functions

- EventList load_workload_csv (const char ∗path)
- const char ∗ event_type_name (EventType t)
- int parse_event_type (const char ∗s, EventType ∗out)

4.19.1 Enumeration Type Documentation

4.19.1.1 EventType

```
enum EventType
```

Enumerator

EV_ARRIVE	
EV_REQ	
EV_REL	
EV_FINISH	
EV_CHECK	

Definition at line 4 of file workload.h.

4.19.2 Function Documentation

4.19.2.1 event_type_name()

```
const char * event_type_name (
            EventType t)
```

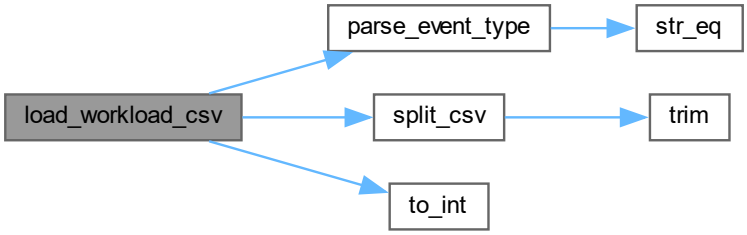
Definition at line 7 of file workload.c.

4.19.2.2 load_workload_csv()

```
EventList load_workload_csv (
            const char * path)
```

Definition at line 27 of file workload.c.

Here is the call graph for this function:



Here is the caller graph for this function:

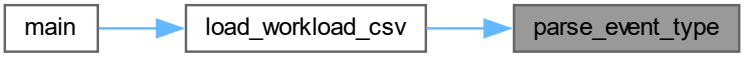
4.19.2.3 parse_event_type()

```
int parse_event_type (
            const char * s,
            EventType * out)
```

Definition at line 18 of file workload.c.

Here is the call graph for this function:

Here is the caller graph for this function:



4.20 workload.h

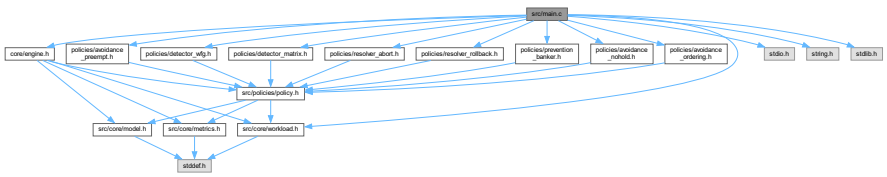
Go to the documentation of this file.

```
#pragma once
#include <stddef.h>

typedef enum { EV_ARRIVE, EV_REQ, EV_REL, EV_FINISH, EV_CHECK } EventType;

typedef struct {
  int time;
  EventType type;
  int pid;
  int rid;
  int amount;
  int hold;
  int seq;
} Event;

typedef struct {
  Event* items;
  size_t len;
} EventList;

EventList load_workload_csv(const char* path);
const char* event_type_name(EventType t);
int parse_event_type(const char* s, EventType* out);
```

4.21 src/main.c File Reference

```
#include "core/engine.h"
#include "core/workload.h"
#include "policies/detector_wfg.h"
#include "policies/detector_matrix.h"
#include "policies/resolver_abort.h"
#include "policies/resolver_rollback.h"
#include "policies/prevention_banker.h"
#include "policies/avoidance_nohold.h"
#include "policies/avoidance_ordering.h"
#include "policies/avoidance_preempt.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

Include dependency graph for main.c:



Functions

- static const char ∗ argval (int ∗i, int argc, char ∗∗argv)
- int main (int argc, char ∗∗argv)

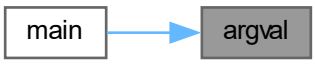
4.21.1 Function Documentation

4.21.1.1 argval()

```
const char * argval (
            int * i,
            int argc,
            char ** argv)  [static]
```

Definition at line 15 of file main.c.

Here is the caller graph for this function:

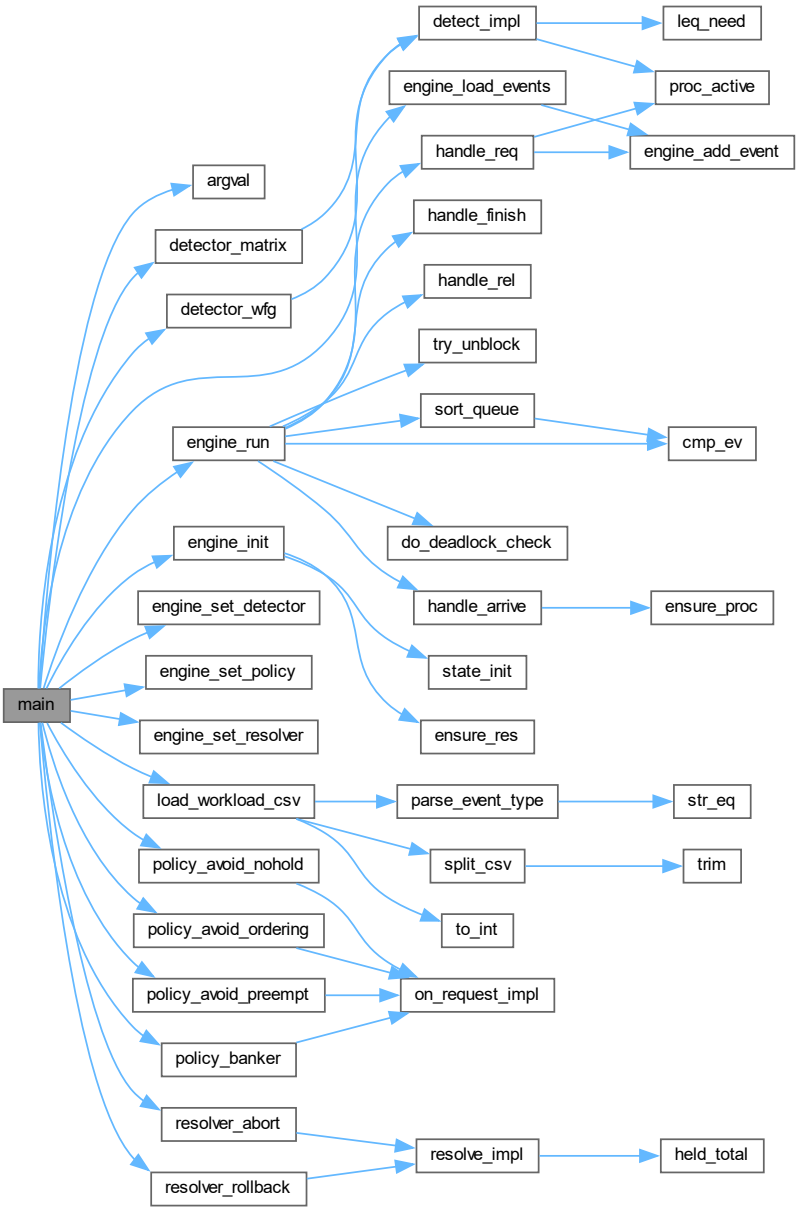


4.21.1.2 main()

```
int main (
            int argc,
            char ** argv)
```

Definition at line 21 of file main.c.

Here is the call graph for this function:



4.22 main.c

Go to the documentation of this file.

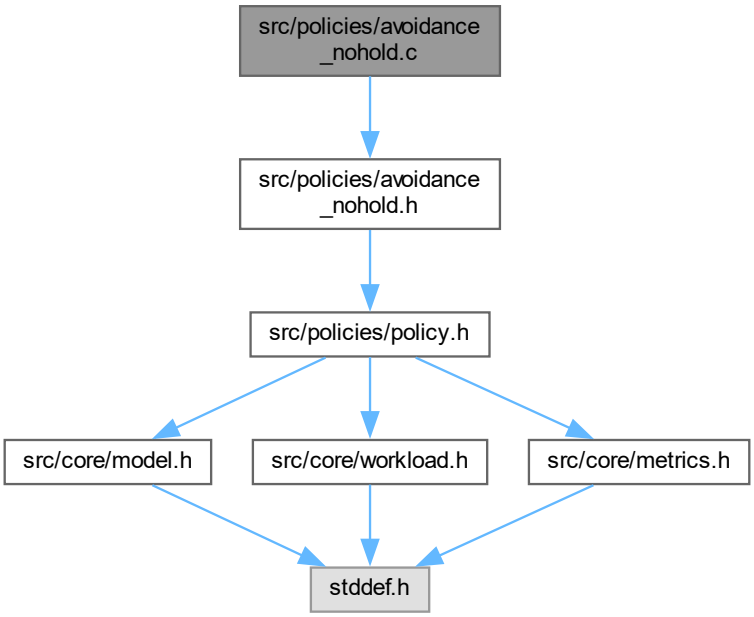
```
00001 #include "core/engine.h"
00002 #include "core/workload.h"
00003 #include "policies/detector_wfg.h"
00004 #include "policies/detector_matrix.h"
00005 #include "policies/resolver_abort.h"
00006 #include "policies/resolver_rollback.h"
00007 #include "policies/prevention_banker.h"
00008 #include "policies/avoidance_nohold.h"
00009 #include "policies/avoidance_ordering.h"
00010 #include "policies/avoidance_preempt.h"
```

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

static const char* argval(int* i, int argc, char** argv) {
  if (*i + 1 >= argc) { fprintf(stderr,"missing value\n"); exit(1); }
  (*i)++;
  return argv[*i];
}

int main(int argc, char** argv) {
  const char* workload = "workloads/demo.csv";
  const char* mode = "detect_abort";
  const char* detector = "matrix";

  for (int i=1;i<argc;i++) {
    if (!strcmp(argv[i],"--workload")) workload = argval(&i,argc,argv);
    else if (!strcmp(argv[i],"--mode")) mode = argval(&i,argc,argv);
    else if (!strcmp(argv[i],"--detector")) detector = argval(&i,argc,argv);
  }

  Engine e;
  engine_init(&e);

  if (!strcmp(detector,"wfg")) engine_set_detector(&e, detector_wfg());
  else engine_set_detector(&e, detector_matrix());

  if (!strcmp(mode,"detect_abort")) {
    engine_set_resolver(&e, resolver_abort());
  } else if (!strcmp(mode,"detect_rollback")) {
    engine_set_resolver(&e, resolver_rollback());
  } else if (!strcmp(mode,"banker")) {
    engine_set_policy(&e, policy_banker());
  } else if (!strcmp(mode,"avoid_nohold")) {
    engine_set_policy(&e, policy_avoid_nohold());
  } else if (!strcmp(mode,"avoid_order")) {
    engine_set_policy(&e, policy_avoid_ordering());
  } else if (!strcmp(mode,"avoid_preempt")) {
    engine_set_policy(&e, policy_avoid_preempt());
  } else {
    fprintf(stderr,"unknown mode\n");
    return 1;
  }

  EventList list = load_workload_csv(workload);
  engine_load_events(&e, &list);
  engine_run(&e, 100000);

  printf("deadlocks=%d aborts=%d rollbacks=%d grants=%d denies=%d defers=%d\n",
    e.m.deadlocks_detected, e.m.aborts, e.m.rollbacks, e.m.grants, e.m.denies, e.m.defers);

  free(list.items);
  return 0;
}
```

4.23 src/policies/avoidance_nohold.c File Reference

```
#include "src/policies/avoidance_nohold.h"
```

Include dependency graph for avoidance_nohold.c:



Functions

- static Decision on_request_impl (const Event ∗e, SystemState ∗s, Metrics ∗m)
- RequestPolicy policy_avoid_nohold (void)

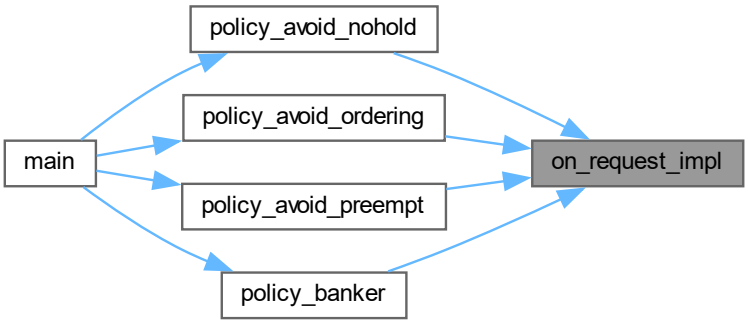
4.23.1 Function Documentation

4.23.1.1 on_request_impl()

```
Decision on_request_impl (
            const Event * e,
            SystemState * s,
            Metrics * m)  [static]
```

Definition at line 3 of file avoidance_nohold.c.

Here is the caller graph for this function:



4.23.1.2 policy_avoid_nohold()

```
RequestPolicy policy_avoid_nohold (
            void )
```

Definition at line 24 of file avoidance_nohold.c.

Here is the call graph for this function:

Here is the caller graph for this function:

4.24 avoidance_nohold.c

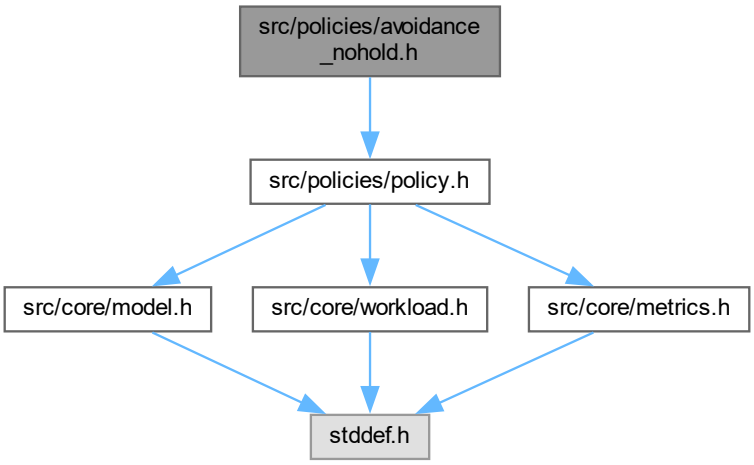
Go to the documentation of this file.

```
#include "src/policies/avoidance_nohold.h"

static Decision on_request_impl(const Event* e, SystemState* s, Metrics* m) {
  int pi = -1, ri = -1;
  for (int i=0;i<s->nprocs;i++) if (s->procs[i].pid == e->pid) { pi=i; break; }
  for (int r=0;r<s->nres;r++) if (s->res[r].rid == e->rid) { ri=r; break; }
  if (pi<0 || ri<0) return DEC_DENY;

  int holding = 0;
  for (int r=0;r<s->nres;r++) if (s->alloc[pi][r] > 0) holding = 1;

  if (holding) {
    m->defers++;
    return DEC_DEFER;
  }

  if (e->amount > s->avail[ri]) { m->defers++; return DEC_DEFER; }
  s->avail[ri] -= e->amount;
  s->alloc[pi][ri] += e->amount;
  m->grants++;
  return DEC_GRANT;
}

RequestPolicy policy_avoid_nohold(void) {
  RequestPolicy p; p.on_request = on_request_impl; return p;
}
```

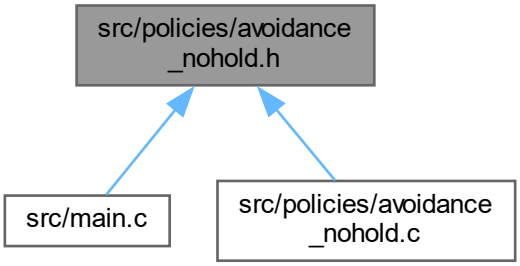
4.25 src/policies/avoidance_nohold.h File Reference

`#include "src/policies/policy.h"`

Include dependency graph for avoidance_nohold.h:



This graph shows which files directly or indirectly include this file:



Functions

- RequestPolicy policy_avoid_nohold (void)

4.25.1 Function Documentation

4.25.1.1 policy_avoid_nohold()

```
RequestPolicy policy_avoid_nohold (
            void )
```

Definition at line 24 of file avoidance_nohold.c.

Here is the call graph for this function:

Here is the caller graph for this function:

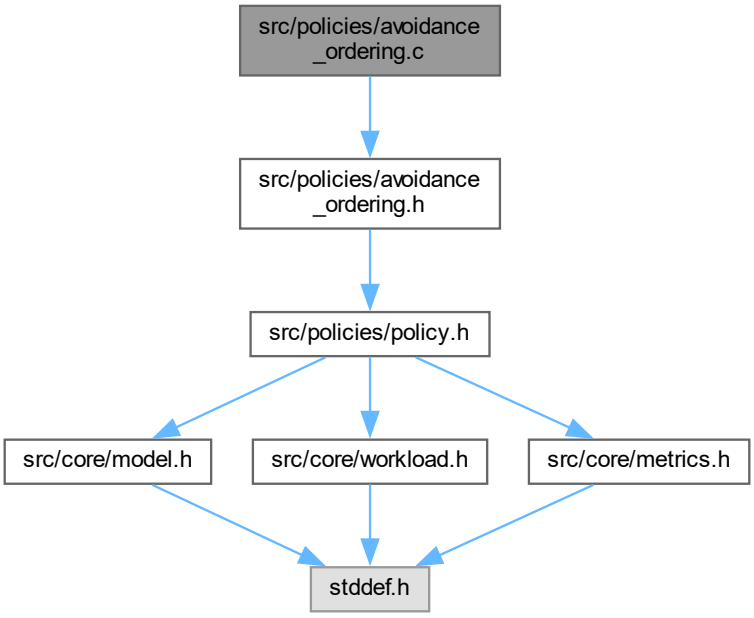
4.26 avoidance_nohold.h

Go to the documentation of this file.

```
00001 #pragma once
00002 #include "src/policies/policy.h"
00003
00004 RequestPolicy policy_avoid_nohold(void);
```

4.27 src/policies/avoidance_ordering.c File Reference

`#include "src/policies/avoidance_ordering.h"`
Include dependency graph for avoidance_ordering.c:



Functions

- static Decision on_request_impl (const Event ∗e, SystemState ∗s, Metrics ∗m)
- RequestPolicy policy_avoid_ordering (void)

4.27.1 Function Documentation

4.27.1.1 on_request_impl()

```
Decision on_request_impl (
            const Event * e,
            SystemState * s,
            Metrics * m)  [static]
```

Definition at line 3 of file avoidance_ordering.c.

4.27.1.2 policy_avoid_ordering()

```
RequestPolicy policy_avoid_ordering (
            void )
```

Definition at line 20 of file avoidance_ordering.c.

Here is the call graph for this function:

Here is the caller graph for this function:

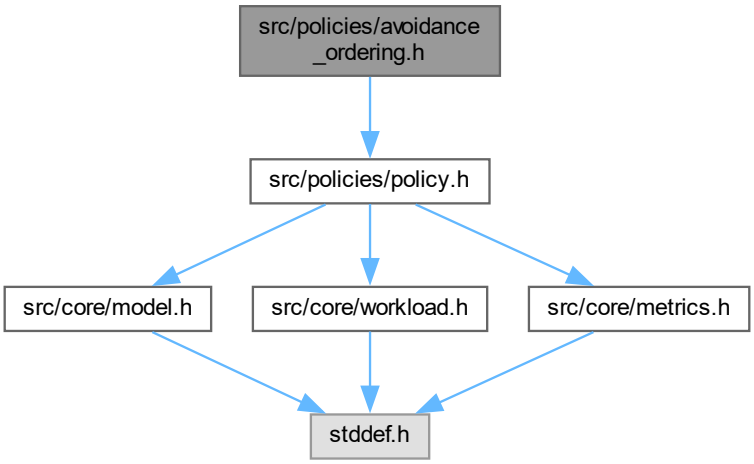
4.28 avoidance_ordering.c

Go to the documentation of this file.

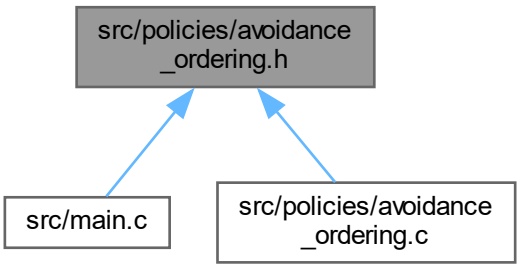
```
00001 #include "src/policies/avoidance_ordering.h"
00002
00003 static Decision on_request_impl(const Event* e, SystemState* s, Metrics* m) {
00004   int pi = -1, ri = -1;
00005   for (int i=0;i<s->nprocs;i++) if (s->procs[i].pid == e->pid) { pi=i; break; }
00006   for (int r=0;r<s->nres;r++) if (s->res[r].rid == e->rid) { ri=r; break; }
00007   if (pi<0 || ri<0) return DEC_DENY;
00008
00009   int maxHeldRid = -1;
00010   for (int r=0;r<s->nres;r++) if (s->alloc[pi][r] > 0 && s->res[r].rid > maxHeldRid) maxHeldRid =
      s->res[r].rid;
00011   if (maxHeldRid != -1 && e->rid < maxHeldRid) { m->denies++; return DEC_DENY; }
00012
00013   if (e->amount > s->avail[ri]) { m->defers++; return DEC_DEFER; }
00014   s->avail[ri] -= e->amount;
00015   s->alloc[pi][ri] += e->amount;
00016   m->grants++;
00017   return DEC_GRANT;
00018 }
00019
00020 RequestPolicy policy_avoid_ordering(void) {
00021   RequestPolicy p; p.on_request = on_request_impl; return p;
00022 }
```

4.29 src/policies/avoidance_ordering.h File Reference

`#include "src/policies/policy.h"`
Include dependency graph for avoidance_ordering.h:



This graph shows which files directly or indirectly include this file:



Functions

- RequestPolicy policy_avoid_ordering (void)

4.29.1 Function Documentation

4.29.1.1 policy_avoid_ordering()

```
RequestPolicy policy_avoid_ordering (
            void )
```

Definition at line 20 of file avoidance_ordering.c.

Here is the call graph for this function:

Here is the caller graph for this function:

4.30 avoidance_ordering.h

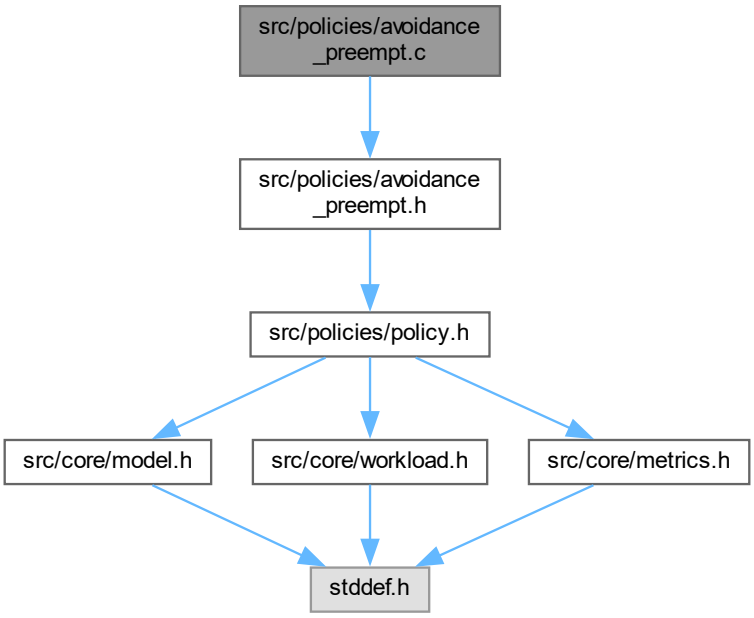
Go to the documentation of this file.

```
#pragma once
#include "src/policies/policy.h"

RequestPolicy policy_avoid_ordering(void);
```

4.31 src/policies/avoidance_preempt.c File Reference

```
#include "src/policies/avoidance_preempt.h"
```

Include dependency graph for avoidance_preempt.c:



Functions

- static Decision on_request_impl (const Event ∗e, SystemState ∗s, Metrics ∗m)
- RequestPolicy policy_avoid_preempt (void)

4.31.1 Function Documentation

4.31.1.1 on_request_impl()

```
Decision on_request_impl (
            const Event * e,
            SystemState * s,
            Metrics * m)  [static]
```

Definition at line 3 of file avoidance_preempt.c.

4.31.1.2 policy_avoid_preempt()

```
RequestPolicy policy_avoid_preempt (
            void )
```

Definition at line 34 of file avoidance_preempt.c.

Here is the call graph for this function:

Here is the caller graph for this function:

4.32 avoidance_preempt.c

Go to the documentation of this file.

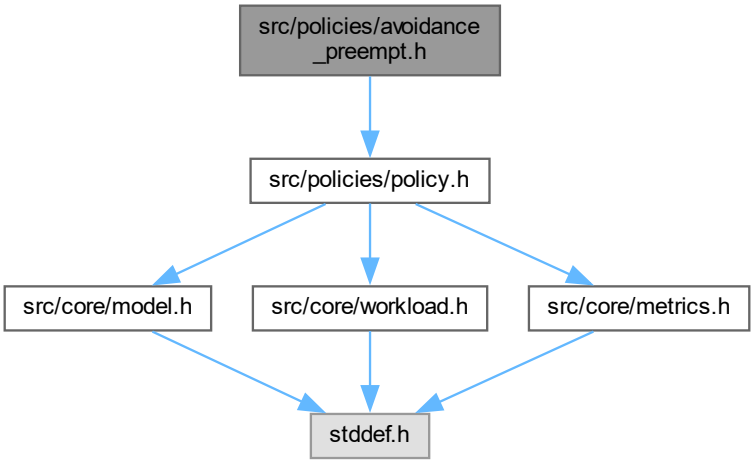
```
#include "src/policies/avoidance_preempt.h"

static Decision on_request_impl(const Event* e, SystemState* s, Metrics* m) {
  int pi = -1, ri = -1;
  for (int i=0;i<s->nprocs;i++) if (s->procs[i].pid == e->pid) { pi=i; break; }
  for (int r=0;r<s->nres;r++) if (s->res[r].rid == e->rid) { ri=r; break; }
  if (pi<0 || ri<0) return DEC_DENY;

  if (e->amount <= s->avail[ri]) {
    s->avail[ri] -= e->amount;
    s->alloc[pi][ri] += e->amount;
    m->grants++;
    return DEC_GRANT;
  }

  for (int r=0;r<s->nres;r++) {
    if (s->alloc[pi][r] > 0) {
      s->avail[r] += s->alloc[pi][r];
      s->alloc[pi][r] = 0;
    }
  }

  if (e->amount <= s->avail[ri]) {
    s->avail[ri] -= e->amount;
    s->alloc[pi][ri] += e->amount;
    m->grants++;
    return DEC_GRANT;
  }

```

```
  m->defers++;
  return DEC_DEFER;
}

RequestPolicy policy_avoid_preempt(void) {
  RequestPolicy p; p.on_request = on_request_impl; return p;
}
```

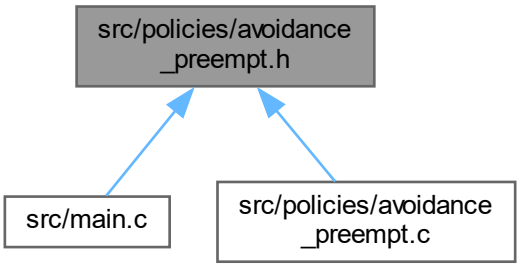
4.33 src/policies/avoidance_preempt.h File Reference

`#include "src/policies/policy.h"`

Include dependency graph for avoidance_preempt.h:



This graph shows which files directly or indirectly include this file:



Functions

- RequestPolicy policy_avoid_preempt (void)

4.33.1 Function Documentation

4.33.1.1 policy_avoid_preempt()

```
RequestPolicy policy_avoid_preempt (
            void )
```

Definition at line 34 of file avoidance_preempt.c.

Here is the call graph for this function:

Here is the caller graph for this function:

4.34 avoidance_preempt.h

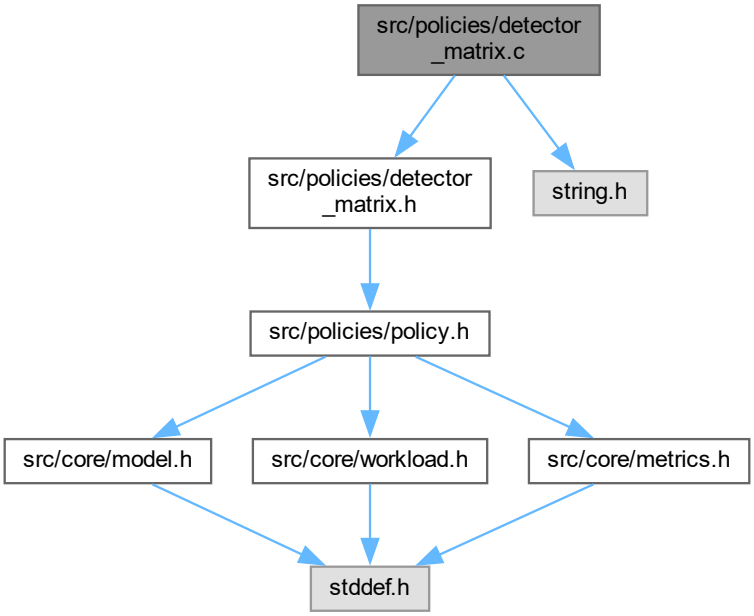
Go to the documentation of this file.

```
00001 #pragma once
00002 #include "src/policies/policy.h"
00003
00004 RequestPolicy policy_avoid_preempt(void);
```

4.35 src/policies/detector_matrix.c File Reference

```
#include "src/policies/detector_matrix.h"
#include <string.h>
```

Include dependency graph for detector_matrix.c:



Functions

- static int leq_need (const int ∗need, const int ∗work, int nres)
- static int detect_impl (const SystemState ∗s, DeadlockSet ∗out, int max_sets)
- Detector detector_matrix (void)

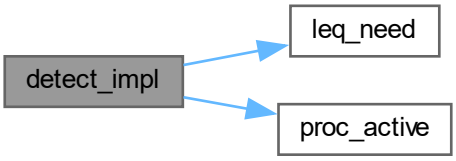
4.35.1 Function Documentation

4.35.1.1 detect_impl()

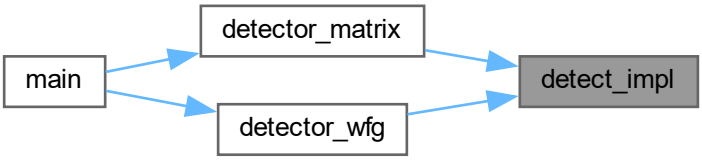
```
int detect_impl (
            const SystemState * s,
            DeadlockSet * out,
            int max_sets)  [static]
```

Definition at line 9 of file detector_matrix.c.

Here is the call graph for this function:



Here is the caller graph for this function:

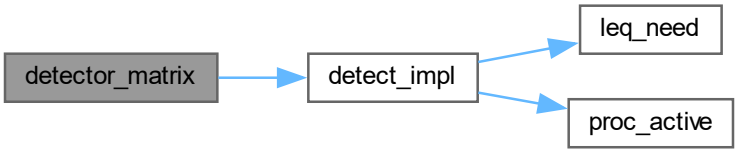


4.35.1.2 detector_matrix()

```
Detector detector_matrix (
            void )
```

Definition at line 40 of file detector_matrix.c.

Here is the call graph for this function:



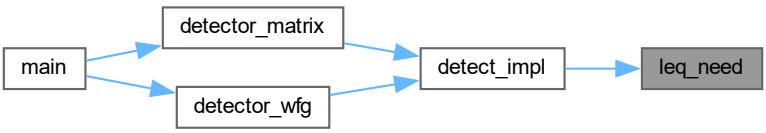
Here is the caller graph for this function:

4.35.1.3 leq_need()

```
int leq_need (
            const int * need,
            const int * work,
            int nres)  [static]
```

Definition at line 4 of file detector_matrix.c.

Here is the caller graph for this function:



4.36 detector_matrix.c

Go to the documentation of this file.

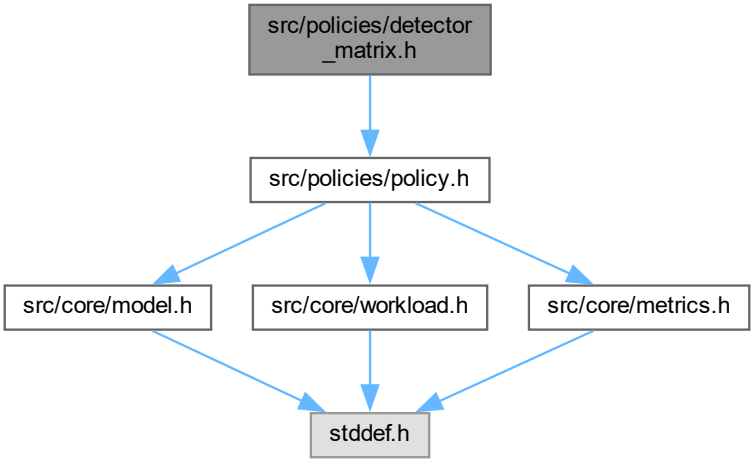
```
#include "src/policies/detector_matrix.h"
#include <string.h>

static int leq_need(const int* need, const int* work, int nres) {
  for (int r=0;r<nres;r++) if (need[r] > work[r]) return 0;
  return 1;
}

static int detect_impl(const SystemState* s, DeadlockSet* out, int max_sets) {
  (void)max_sets;
  int work[MAX_RES];
  int finish[MAX_PROCS];
  memcpy(work, s->avail, sizeof(int)*s->nres);
  memset(finish, 0, sizeof(finish));

  int progress = 1;
  while (progress) {
    progress = 0;
    for (int i=0;i<s->nprocs;i++) {
      if (!proc_active(s->procs[i].state)) { finish[i]=1; continue; }
      if (finish[i]) continue;
      if (leq_need(s->req[i], work, s->nres)) {
        for (int r=0;r<s->nres;r++) work[r] += s->alloc[i][r];
```

```
        finish[i] = 1;
        progress = 1;
      }
    }
  }

  DeadlockSet d; d.len = 0;
  for (int i=0;i<s->nprocs;i++) {
    if (!proc_active(s->procs[i].state)) continue;
    if (!finish[i]) d.pids[d.len++] = s->procs[i].pid;
  }
  if (d.len == 0) return 0;
  out[0] = d;
  return 1;
}

Detector detector_matrix(void) {
  Detector d; d.detect = detect_impl; return d;
}
```

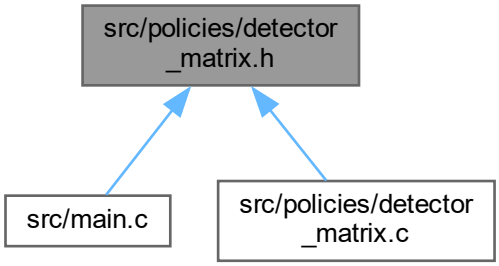
4.37 src/policies/detector_matrix.h File Reference

`#include "src/policies/policy.h"`

Include dependency graph for detector_matrix.h:



This graph shows which files directly or indirectly include this file:



Functions

- Detector detector_matrix (void)

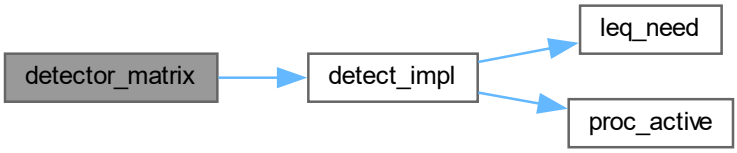
4.37.1 Function Documentation

4.37.1.1 detector_matrix()

```
Detector detector_matrix (
            void )
```

Definition at line 40 of file detector_matrix.c.

Here is the call graph for this function:



Here is the caller graph for this function:

4.38 detector_matrix.h

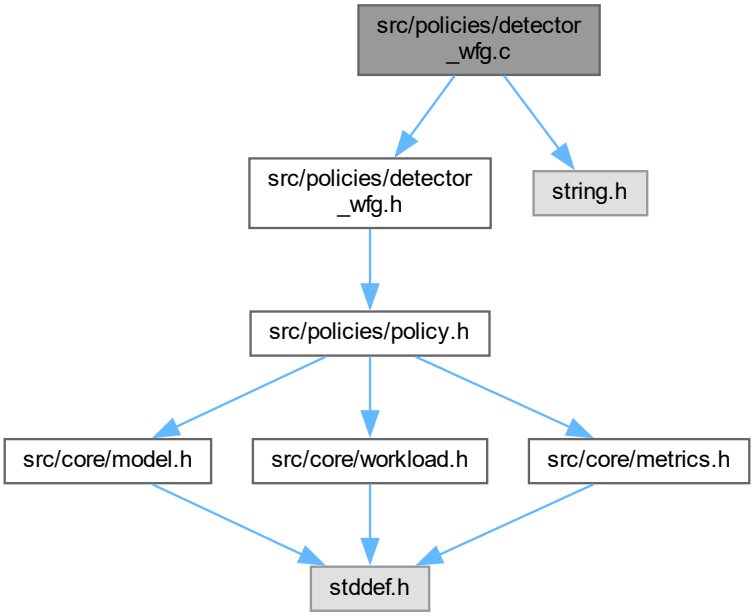
Go to the documentation of this file.

```
00001 #pragma once
00002 #include "src/policies/policy.h"
00003
00004 Detector detector_matrix(void);
```

4.39 src/policies/detector_wfg.c File Reference

```
#include "src/policies/detector_wfg.h"
#include <string.h>
```

Include dependency graph for detector_wfg.c:



Functions

- static int detect_impl (const SystemState *s, DeadlockSet *out, int max_sets)
- Detector detector_wfg (void)

4.39.1 Function Documentation

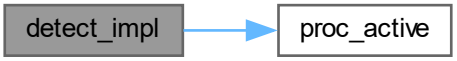
4.39.1.1 detect_impl()

```
int detect_impl (
            const SystemState * s,
```

```
            DeadlockSet * out,
            int max_sets)  [static]
```

Definition at line 4 of file detector_wfg.c.

Here is the call graph for this function:

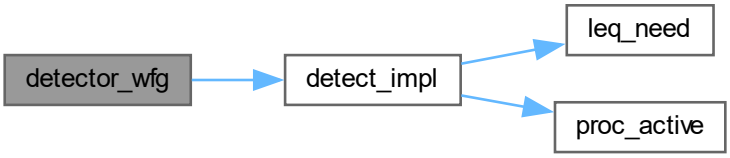


4.39.1.2 detector_wfg()

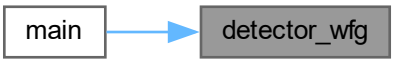
```
Detector detector_wfg (
            void )
```

Definition at line 73 of file detector_wfg.c.

Here is the call graph for this function:



Here is the caller graph for this function:



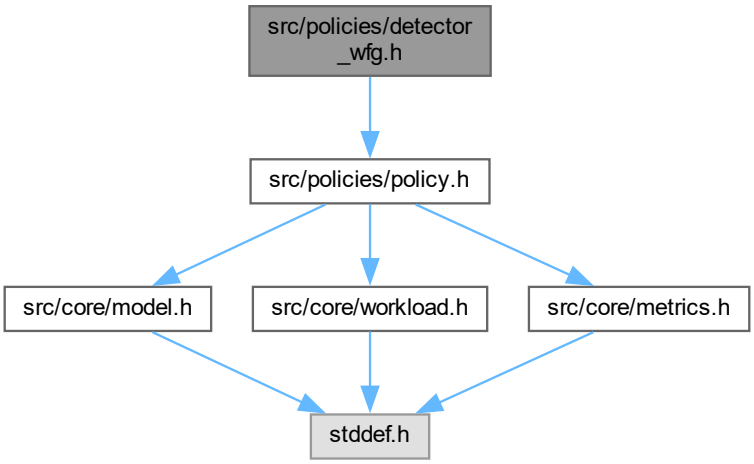
4.40 detector_wfg.c

Go to the documentation of this file.

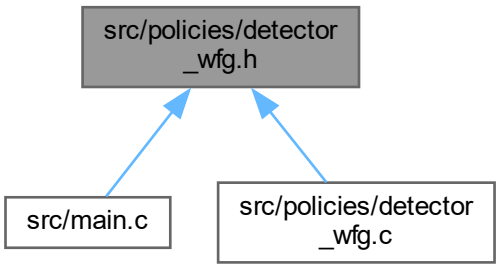
```
#include "src/policies/detector_wfg.h"
#include <string.h>

static int detect_impl(const SystemState* s, DeadlockSet* out, int max_sets) {
  int n = s->nprocs;
  int g[MAX_PROCS][MAX_PROCS];
  memset(g, 0, sizeof(g));

  for (int pi=0; pi<n; pi++) {
    if (!proc_active(s->procs[pi].state)) continue;
    for (int r=0; r<s->nres; r++) {
      if (s->req[pi][r] <= 0) continue;
      for (int qi=0; qi<n; qi++) {
        if (qi==pi) continue;
        if (!proc_active(s->procs[qi].state)) continue;
        if (s->alloc[qi][r] > 0) g[pi][qi] = 1;
      }
    }
  }

  int color[MAX_PROCS];
  int stack[MAX_PROCS];
  int sp = 0;
  memset(color, 0, sizeof(color));

  int found = 0;

  for (int start=0; start<n && found<max_sets; start++) {
    if (!proc_active(s->procs[start].state)) continue;
    if (color[start] != 0) continue;

    int iters[MAX_PROCS];
    memset(iters, 0, sizeof(iters));
    int parent[MAX_PROCS];
    for (int i=0;i<MAX_PROCS;i++) parent[i] = -1;

    int cur = start;
    while (1) {
      if (color[cur] == 0) {
        color[cur] = 1;
        stack[sp++] = cur;
      }

      int advanced = 0;
      for (int v=iters[cur]; v<n; v++) {
        iters[cur] = v+1;
        if (!g[cur][v]) continue;
        if (color[v] == 0) { parent[v] = cur; cur = v; advanced = 1; break; }
        if (color[v] == 1) {
          DeadlockSet d; d.len = 0;
          int k = sp-1;
          while (k>=0 && stack[k] != v) { k--; }
          if (k>=0) {
            for (int i=k;i<sp;i++) d.pids[d.len++] = s->procs[stack[i]].pid;
            out[found++] = d;
          }
        }
      }
      if (found>=max_sets) break;
      if (advanced) continue;

      if (sp == 0) break;
      int popped = stack[--sp];
      color[popped] = 2;
      if (sp == 0) break;
      cur = stack[sp-1];
    }
  }

  return found;
}

Detector detector_wfg(void) {
  Detector d; d.detect = detect_impl; return d;
}
```

4.41 src/policies/detector_wfg.h File Reference

`#include "src/policies/policy.h"`
Include dependency graph for detector_wfg.h:



This graph shows which files directly or indirectly include this file:



Functions

- Detector detector_wfg (void)

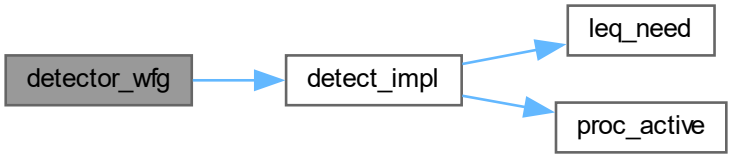
4.41.1 Function Documentation

4.41.1.1 detector_wfg()

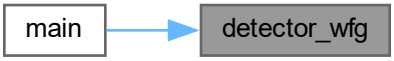
```
Detector detector_wfg (
            void )
```

Definition at line 73 of file detector_wfg.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.42 detector_wfg.h

Go to the documentation of this file.

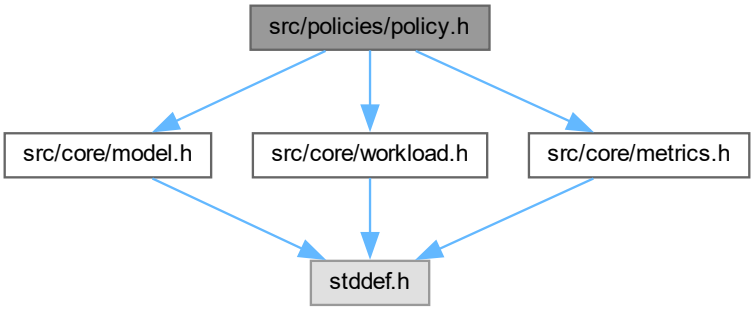
```
00001 #pragma once
00002 #include "src/policies/policy.h"
00003
00004 Detector detector_wfg(void);
```

4.43 src/policies/policy.h File Reference

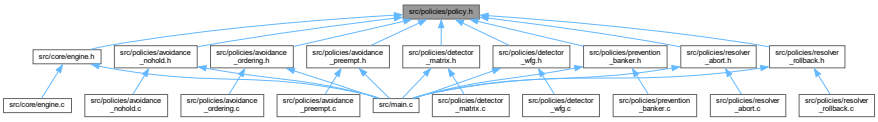
```
#include "src/core/model.h"
#include "src/core/workload.h"
```

```
#include "src/core/metrics.h"
```

Include dependency graph for policy.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct DeadlockSet
- struct Detector
- struct Resolver
- struct RequestPolicy

Enumerations

- enum Decision { DEC_GRANT , DEC_DENY , DEC_DEFER }

4.43.1 Enumeration Type Documentation

4.43.1.1 Decision

```
enum Decision
```

Enumerator

	DEC_GRANT
	DEC_DENY
	DEC_DEFER

Definition at line 6 of file policy.h.

4.44 policy.h

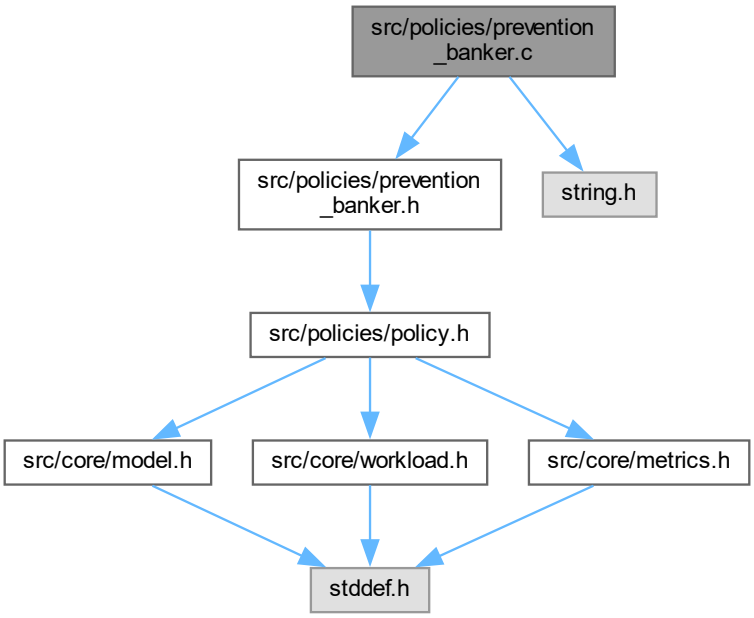
Go to the documentation of this file.

```
#pragma once
#include "src/core/model.h"
#include "src/core/workload.h"
#include "src/core/metrics.h"

typedef enum { DEC_GRANT, DEC_DENY, DEC_DEFER } Decision;

typedef struct {
  int pids[MAX_PROCS];
  int len;
} DeadlockSet;

typedef struct {
  int (*detect)(const SystemState* s, DeadlockSet* out_sets, int max_sets);
} Detector;

typedef struct {
  void (*resolve)(const DeadlockSet* d, SystemState* s, int now, Metrics* m);
} Resolver;

typedef struct {
  Decision (*on_request)(const Event* e, SystemState* s, Metrics* m);
} RequestPolicy;
```

4.45 src/policies/prevention_banker.c File Reference

```
#include "src/policies/prevention_banker.h"
#include <string.h>
```

Include dependency graph for prevention_banker.c:



Functions

- static int safety_check (const SystemState ∗s)
- static Decision on_request_impl (const Event ∗e, SystemState ∗s, Metrics ∗m)
- RequestPolicy policy_banker (void)

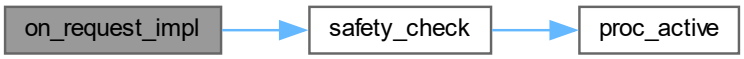
4.45.1 Function Documentation

4.45.1.1 on_request_impl()

```
Decision on_request_impl (
            const Event * e,
            SystemState * s,
            Metrics * m)  [static]
```

Definition at line 34 of file prevention_banker.c.

Here is the call graph for this function:

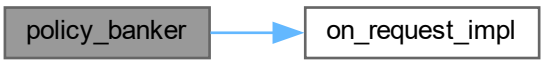


4.45.1.2 policy_banker()

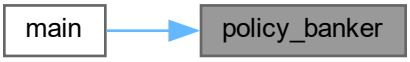
```
RequestPolicy policy_banker (
            void )
```

Definition at line 59 of file prevention_banker.c.

Here is the call graph for this function:



Here is the caller graph for this function:



4.45.1.3 safety_check()

```
int safety_check (
            const SystemState * s)  [static]
```

Definition at line 4 of file prevention_banker.c.

Here is the call graph for this function:

Here is the caller graph for this function:

4.46 prevention_banker.c

Go to the documentation of this file.

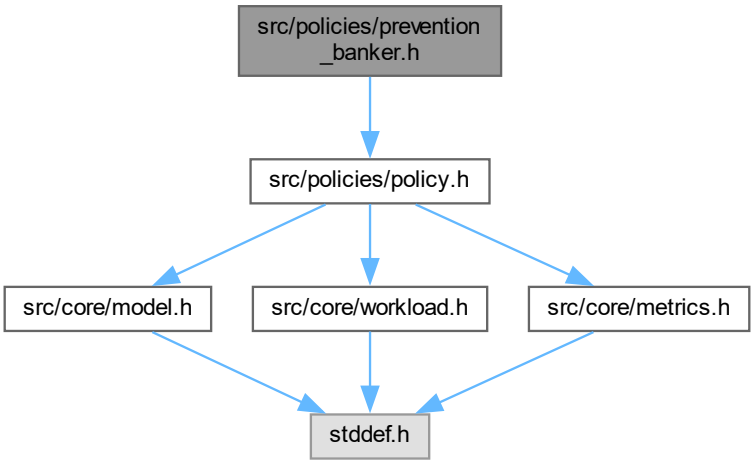
```
#include "src/policies/prevention_banker.h"
#include <string.h>

static int safety_check(const SystemState* s) {
  int work[MAX_RES];
  int finish[MAX_PROCS];
  memcpy(work, s->avail, sizeof(int)*s->nres);
```

```c
  memset(finish, 0, sizeof(finish));

  int progress = 1;
  while (progress) {
    progress = 0;
    for (int i=0;i<s->nprocs;i++) {
      if (!proc_active(s->procs[i].state)) { finish[i]=1; continue; }
      if (finish[i]) continue;

      int ok = 1;
      for (int r=0;r<s->nres;r++) {
        int need = s->maxc[i][r] - s->alloc[i][r];
        if (need > work[r]) { ok = 0; break; }
      }
      if (ok) {
        for (int r=0;r<s->nres;r++) work[r] += s->alloc[i][r];
        finish[i] = 1;
        progress = 1;
      }
    }
  }

  for (int i=0;i<s->nprocs;i++) if (proc_active(s->procs[i].state) && !finish[i]) return 0;
  return 1;
}

static Decision on_request_impl(const Event* e, SystemState* s, Metrics* m) {
  int pi = -1, ri = -1;
  for (int i=0;i<s->nprocs;i++) if (s->procs[i].pid == e->pid) { pi=i; break; }
  for (int r=0;r<s->nres;r++) if (s->res[r].rid == e->rid) { ri=r; break; }
  if (pi<0 || ri<0) return DEC_DENY;

  int need = s->maxc[pi][ri] - s->alloc[pi][ri];
  if (e->amount > need) { m->denies++; return DEC_DENY; }
  if (e->amount > s->avail[ri]) { m->defers++; return DEC_DEFER; }

  s->avail[ri] -= e->amount;
  s->alloc[pi][ri] += e->amount;

  int safe = safety_check(s);
  if (!safe) {
    s->alloc[pi][ri] -= e->amount;
    s->avail[ri] += e->amount;
    m->denies++;
    return DEC_DENY;
  }

  m->grants++;
  return DEC_GRANT;
}

RequestPolicy policy_banker(void) {
  RequestPolicy p; p.on_request = on_request_impl; return p;
}
```

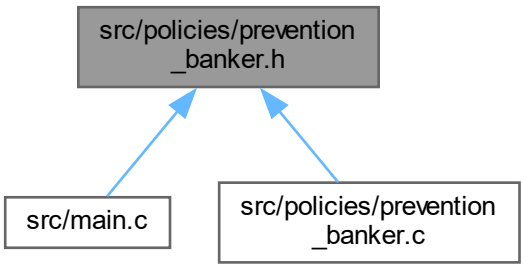
4.47 src/policies/prevention_banker.h File Reference

```
#include "src/policies/policy.h"
```

Include dependency graph for prevention_banker.h:



This graph shows which files directly or indirectly include this file:



Functions

- RequestPolicy policy_banker (void)

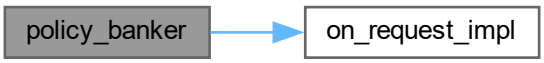
4.47.1 Function Documentation

4.47.1.1 policy_banker()

```
RequestPolicy policy_banker (
            void )
```

Definition at line 59 of file prevention_banker.c.

Here is the call graph for this function:



Here is the caller graph for this function:

4.48 prevention_banker.h

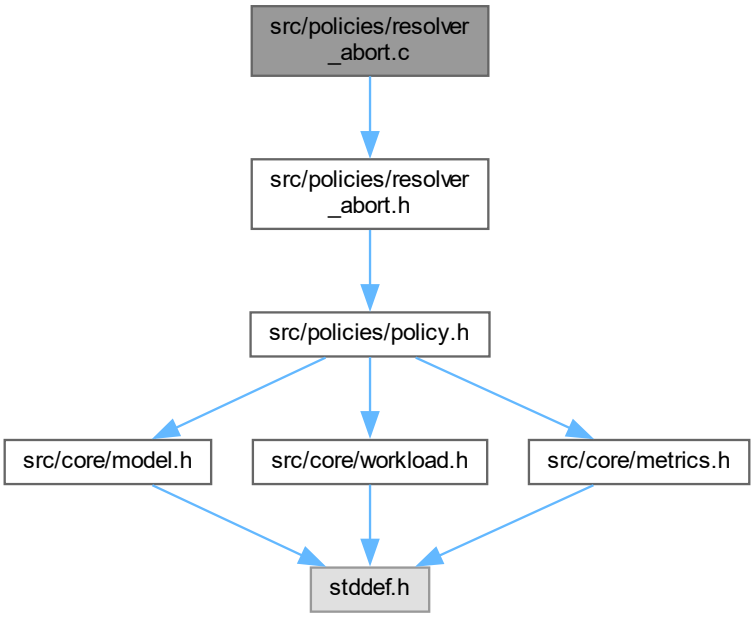
Go to the documentation of this file.

```
00001 #pragma once
00002 #include "src/policies/policy.h"
00003
00004 RequestPolicy policy_banker(void);
```

4.49 src/policies/resolver_abort.c File Reference

```
#include "src/policies/resolver_abort.h"
```

Include dependency graph for resolver_abort.c:



Functions

- static int held_total (const SystemState ∗s, int pi)
- static void resolve_impl (const DeadlockSet ∗d, SystemState ∗s, int now, Metrics ∗m)
- Resolver resolver_abort (void)

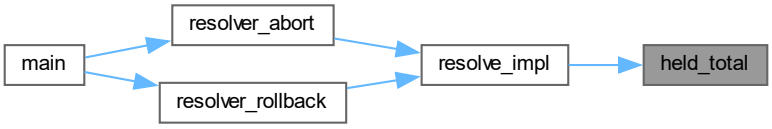
4.49.1 Function Documentation

4.49.1.1 held_total()

```
int held_total (
            const SystemState * s,
            int pi)  [static]
```

Definition at line 3 of file resolver_abort.c.

Here is the caller graph for this function:



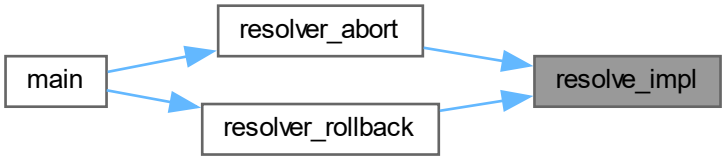
4.49.1.2 resolve_impl()

```
void resolve_impl (
            const DeadlockSet * d,
            SystemState * s,
            int now,
            Metrics * m)  [static]
```

Definition at line 9 of file resolver_abort.c.

Here is the call graph for this function:

Here is the caller graph for this function:



4.49.1.3 resolver_abort()

```
Resolver resolver_abort (
            void )
```

Definition at line 34 of file resolver_abort.c.

Here is the call graph for this function:

Here is the caller graph for this function:

4.50 resolver_abort.c

Go to the documentation of this file.

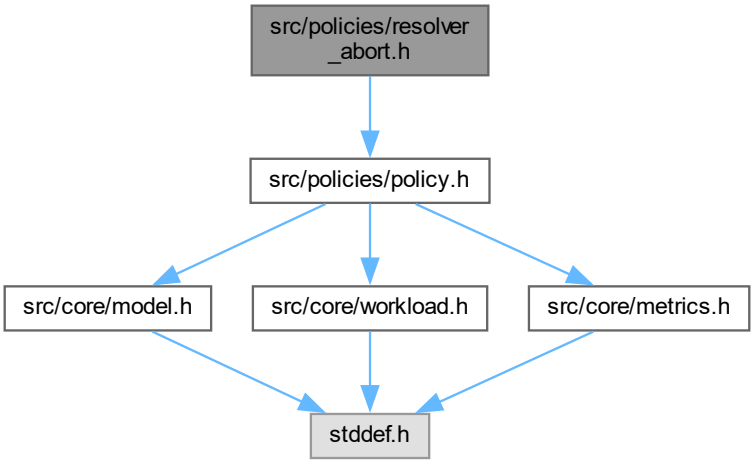
```
00001 #include "src/policies/resolver_abort.h"
00002
00003 static int held_total(const SystemState* s, int pi) {
00004   int sum = 0;
00005   for (int r=0;r<s->nres;r++) if (s->alloc[pi][r] > 0) sum += s->alloc[pi][r];
00006   return sum;
00007 }
00008
00009 static void resolve_impl(const DeadlockSet* d, SystemState* s, int now, Metrics* m) {
00010   (void)now;
00011   if (d->len == 0) return;
00012
00013   int victim_pi = -1;
00014   int best = -1;
00015   for (int k=0;k<d->len;k++) {
00016     int pid = d->pids[k];
00017     int pi = -1;
00018     for (int i=0;i<s->nprocs;i++) if (s->procs[i].pid == pid) { pi=i; break; }
00019     if (pi<0) continue;
00020     int h = held_total(s, pi);
00021     if (h > best) { best = h; victim_pi = pi; }
00022   }
00023   if (victim_pi < 0) return;
00024
00025   s->procs[victim_pi].state = PS_ABORTED;
00026   for (int r=0;r<s->nres;r++) {
00027     s->avail[r] += s->alloc[victim_pi][r];
00028     s->alloc[victim_pi][r] = 0;
00029     s->req[victim_pi][r] = 0;
```

```
  }
  m->aborts++;
}

Resolver resolver_abort(void) {
  Resolver r; r.resolve = resolve_impl; return r;
}
```

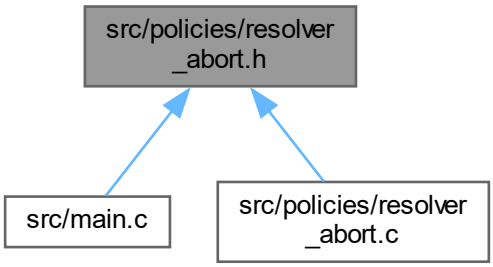
4.51 src/policies/resolver_abort.h File Reference

`#include "src/policies/policy.h"`

Include dependency graph for resolver_abort.h:



This graph shows which files directly or indirectly include this file:



Functions

- Resolver resolver_abort (void)

4.51.1 Function Documentation

4.51.1.1 resolver_abort()

```
Resolver resolver_abort (
            void )
```

Definition at line 34 of file resolver_abort.c.

Here is the call graph for this function:

Here is the caller graph for this function:

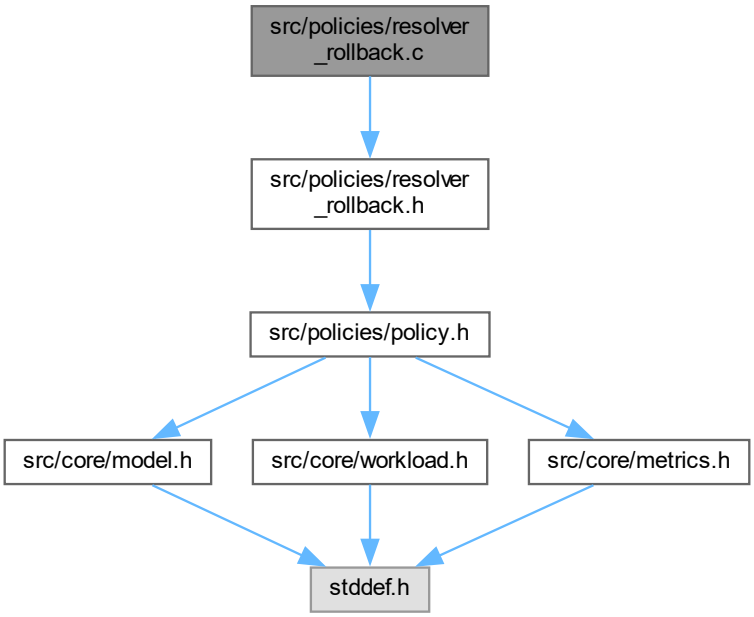
4.52 resolver_abort.h

Go to the documentation of this file.

```
00001 #pragma once
00002 #include "src/policies/policy.h"
00003
00004 Resolver resolver_abort(void);
```

4.53 src/policies/resolver_rollback.c File Reference

`#include "src/policies/resolver_rollback.h"`
Include dependency graph for resolver_rollback.c:



Functions

- static void resolve_impl (const DeadlockSet ∗d, SystemState ∗s, int now, Metrics ∗m)
- Resolver resolver_rollback (void)

4.53.1 Function Documentation

4.53.1.1 resolve_impl()

```
void resolve_impl (
            const DeadlockSet * d,
            SystemState * s,
            int now,
            Metrics * m)  [static]
```

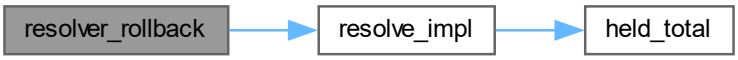
Definition at line 3 of file resolver_rollback.c.

4.53.1.2 resolver_rollback()

```
Resolver resolver_rollback (
            void )
```

Definition at line 23 of file resolver_rollback.c.

Here is the call graph for this function:



Here is the caller graph for this function:

4.54 resolver_rollback.c

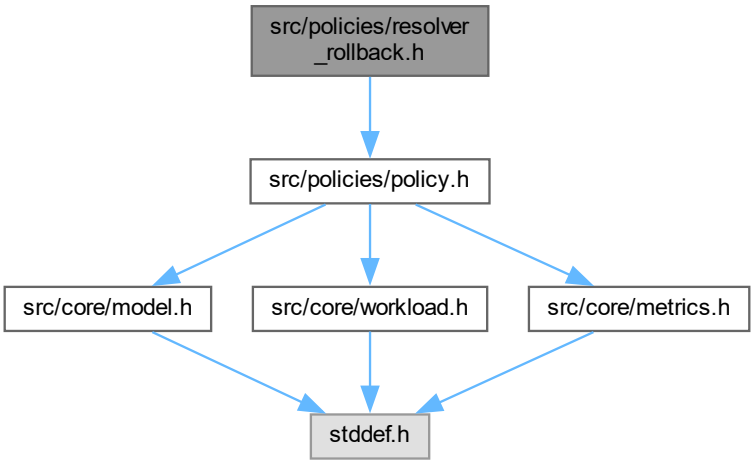
Go to the documentation of this file.

```
#include "src/policies/resolver_rollback.h"

static void resolve_impl(const DeadlockSet* d, SystemState* s, int now, Metrics* m) {
  (void)now;
  if (d->len == 0) return;

  int pid = d->pids[0];
  int pi = -1;
  for (int i=0;i<s->nprocs;i++) if (s->procs[i].pid == pid) { pi=i; break; }
  if (pi<0) return;

  s->procs[pi].state = PS_ROLLED_BACK;
  for (int r=0;r<s->nres;r++) {
    if (s->alloc[pi][r] > 0) {
      s->avail[r] += s->alloc[pi][r];
      s->alloc[pi][r] = 0;
    }
    s->req[pi][r] = 0;
  }
  m->rollbacks++;
}

Resolver resolver_rollback(void) {
  Resolver r; r.resolve = resolve_impl; return r;
}
```

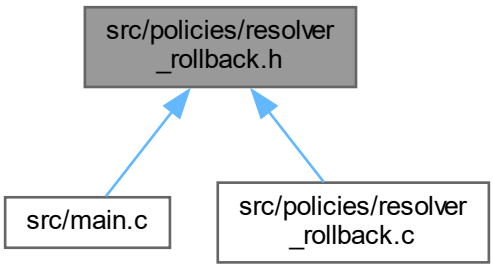
4.55 src/policies/resolver_rollback.h File Reference

```
#include "src/policies/policy.h"
```

Include dependency graph for resolver_rollback.h:



This graph shows which files directly or indirectly include this file:



Functions

- Resolver resolver_rollback (void)

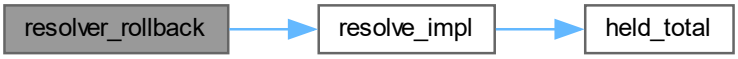
4.55.1 Function Documentation

4.55.1.1 resolver_rollback()

```
Resolver resolver_rollback (
            void )
```

Definition at line 23 of file resolver_rollback.c.

Here is the call graph for this function:



Here is the caller graph for this function:

4.56 resolver_rollback.h

Go to the documentation of this file.

```
00001 #pragma once
00002 #include "src/policies/policy.h"
00003
00004 Resolver resolver_rollback(void);
```